INDUSTRIAL ASSIGNMENT- 5 Database Queries
STUDENT INTERNSHIP AND PLACEMENT MANAGEMENT PORTAL

Episode No.	Episode Name	Tasks	Queries
EP01	User Management	4	36
EP02	Student Profile Management	4	27
EP03	Company and Opportunity Management	4	29
EP04	Application Management	4	30
EP05	Interview and Offer Management	4	26
EP06	Dashboard and Reports	4	28
EP07	Search and Filter	4	20

Document Scope

This document contains SQL database queries for all 28 tasks (T01–T28) across Episodes EP01–EP07 of the Student Internship and Placement Management Portal, based on the requirements (Industrial Assignment 2) and the user stories (Industrial Assignments 3 and 4). The SQL is organized task-wise and written in MySQL 8.0 style. It contains 196 queries in total. Every CREATE TABLE statement includes both created_at and updated_at timestamp attributes. The 16 tables created by these queries are described attribute by attribute in the Entity and Attribute Dictionary at the end of this document.

Conventions Used

- Each table is created once, in the first task that needs it. Task numbers in brackets such as "(run after Task T13)" show that a query uses a table created in a later task.
- A query that contains more than one SQL statement (for example an INSERT into a history table followed by an UPDATE) is executed as one unit of work for that action.
- Sample data uses fictional names and values. Identifiers in the queries (user_id 1–9, student_id 1–2, company_id 1–2, opportunity_id 1–5, application_id 1–4) refer to the sample rows shown in the Sample Data Reference below.
- Passwords are stored with SHA2(password, 256); plain-text passwords are never stored.
- Role checks are written into the WHERE clauses so that each role sees or changes only the records permitted in its Permission Matrix.
- Deleted users and deleted opportunities are soft-deleted (status = 'Deleted') so that their history is preserved.

TASK INDEX

Episode	Task ID	Task Title	Query Count
EP01	T01	Register User	10
EP01	T02	User Login	8
EP01	T03	User Logout	6
EP01	T04	Manage User Access	12
EP02	T05	Create Student Profile	8
EP02	T06	View Student Profile	5
EP02	T07	Edit Student Profile	6
EP02	T08	Upload and Manage Resume	8
EP03	T09	Create Company Profile	7
EP03	T10	Post Opportunity	8
EP03	T11	Edit Opportunity	6
EP03	T12	Close or Delete Opportunity	8
EP04	T13	Apply for Opportunity	10
EP04	T14	View Application Details	6
EP04	T15	Update Application Status	8
EP04	T16	Withdraw Application	6
EP05	T17	Schedule Interview	7
EP05	T18	View Interview Schedule	5
EP05	T19	Release Offer	7
EP05	T20	Respond to Offer	7
EP06	T21	View Dashboard	8
EP06	T22	View Opportunity Summary	6
EP06	T23	View Placement Summary	6
EP06	T24	Generate Reports	8
EP07	T25	Search Opportunities	5
EP07	T26	Filter Opportunities	5
EP07	T27	Search Student Records	5
EP07	T28	Filter Applications	5

SAMPLE DATA REFERENCE

user_id	Name	Role	Linked Record
1	Sana Khan	Administrator	–
2	Priya Shah	Student	student_id 1 (DCSE2023002, CGPA 8.60)
3	Rohan Mehta	Student	student_id 2 (DCSE2023003, CGPA 7.10)
4	Karan Joshi	Company Recruiter	company_id 1 (TechNova Solutions)
5	Neha Patel	Placement Officer	–
6	Dr. Amit Verma	Faculty Coordinator (CSE)	–
7	Anjali Nair	Company Recruiter	company_id 2 (BrightPath Infotech)
8	Vikram Rao	Company Recruiter (test, rejected)	–
9	Meera Iyer	Faculty Coordinator (test)	–

Opportunity	Title	Company	Type
1	Java Developer Intern	TechNova Solutions	Internship
2	Software Engineer Trainee	TechNova Solutions	Placement
3	Web Developer Intern	BrightPath Infotech	Internship
4	Associate Developer	BrightPath Infotech	Placement
5	Test Posting - Duplicate Entry	BrightPath Infotech	Internship (test)

EP01 - USER MANAGEMENT

TASK T01 - REGISTER USER

Task ID	T01
User Story ID	EP01-US01
Task Title	Register User

DATABASE QUERIES

Query 1 - Create User Registration Table

```
CREATE TABLE users (
 user_id INT PRIMARY KEY AUTO_INCREMENT,
 full_name VARCHAR(100) NOT NULL,
 email VARCHAR(100) NOT NULL UNIQUE,
 phone VARCHAR(15) NOT NULL UNIQUE,
 password_hash VARCHAR(255) NOT NULL,
 role VARCHAR(30) NOT NULL,
 status VARCHAR(20) DEFAULT 'Pending',
 failed_attempts INT DEFAULT 0,
 locked_until TIMESTAMP NULL,
 address VARCHAR(255),
 employee_id VARCHAR(50),
 designation VARCHAR(100),
 department VARCHAR(100),
 joining_date DATE,
 enrollment_no VARCHAR(30) UNIQUE,
 branch VARCHAR(50),
 semester INT,
 company_name VARCHAR(150),
 company_website VARCHAR(150),
 appointment_date DATE,
 scope_of_access VARCHAR(255),
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
 );
```

Query 2 - Insert Administrator Registration

```
INSERT INTO users (full_name, email, phone, password_hash, role, status,
  address, employee_id,
 designation, joining_date, appointment_date, scope_of_access)
VALUES ('Sana Khan', 'sana.khan@college.edu', '9876500001', SHA2('Sana@123',
  256), 'Administrator',
 'Active', 'Ahmedabad, Gujarat', 'ADM101', 'System Administrator',
```

```
  '2026-06-01', '2026-06-01',
'All Modules');
```

Query 3 - Insert Student Registration - Priya Shah

```
INSERT INTO users (full_name, email, phone, password_hash, role, status,
  address, enrollment_no,
 branch, semester)
VALUES ('Priya Shah', 'priya.shah@college.edu', '9876500002',
  SHA2('Priya@123', 256), 'Student',
 'Active', 'Ahmedabad, Gujarat', 'DCSE2023002', 'CSE', 5);
```

Query 4 - Insert Student Registration - Rohan Mehta

```
INSERT INTO users (full_name, email, phone, password_hash, role, status,
  address, enrollment_no,
 branch, semester)
VALUES ('Rohan Mehta', 'rohan.mehta@college.edu', '9876500003',
  SHA2('Rohan@123', 256), 'Student',
 'Active', 'Surat, Gujarat', 'DCSE2023003', 'CSE', 5);
```

Query 5 - Insert Company Recruiter Registration - TechNova

```
INSERT INTO users (full_name, email, phone, password_hash, role, status,
  designation, company_name,
 company_website)
VALUES ('Karan Joshi', 'karan.joshi@technova.com', '9876500004',
  SHA2('Karan@123', 256),
 'Company Recruiter', 'Pending', 'HR Manager', 'TechNova Solutions',
   'www.technova.com');
```

Query 6 - Insert Placement Officer Registration

```
INSERT INTO users (full_name, email, phone, password_hash, role, status,
  address, employee_id,
 designation, department, joining_date)
VALUES ('Neha Patel', 'neha.patel@college.edu', '9876500005', SHA2('Neha@123',
  256),
 'Placement Officer', 'Pending', 'Ahmedabad, Gujarat', 'PO101', 'Placement
   Officer',
 'Training and Placement Cell', '2026-06-15');
```

Query 7 - Insert Faculty Coordinator Registration

```
INSERT INTO users (full_name, email, phone, password_hash, role, status,
  address, employee_id,
 designation, department, joining_date)
VALUES ('Dr. Amit Verma', 'amit.verma@college.edu', '9876500006',
```

```
 SHA2('Amit@123', 256),
'Faculty Coordinator', 'Pending', 'Ahmedabad, Gujarat', 'FC101', 'Head of
  Department',
'Computer Science and Engineering', '2026-06-15');
```

Query 8 - Insert Company Recruiter Registration - BrightPath

```
INSERT INTO users (full_name, email, phone, password_hash, role, status,
  designation, company_name,
 company_website)
VALUES ('Anjali Nair', 'anjali.nair@brightpath.com', '9876500007',
  SHA2('Anjali@123', 256),
 'Company Recruiter', 'Pending', 'Talent Acquisition Lead', 'BrightPath
  Infotech',
'www.brightpath.in');
```

Query 9 - Check Duplicate Email, Phone or Enrollment Number

```
SELECT user_id, full_name, email, phone, enrollment_no, role
FROM users
WHERE email = 'priya.shah@college.edu'
 OR phone = '9876500002'
 OR enrollment_no = 'DCSE2023002';
```

Query 10 - Display Registered Users

```
SELECT user_id, full_name, email, phone, role, status, created_at, updated_at
FROM users
ORDER BY created_at DESC;
```

EP01 - USER MANAGEMENT

TASK T02 - USER LOGIN

Task ID	T02
User Story ID	EP01-US02
Task Title	User Login

DATABASE QUERIES

Query 11 - Create Login Session Table

```
CREATE TABLE login_sessions (
 session_id INT PRIMARY KEY AUTO_INCREMENT,
 user_id INT NOT NULL,
 session_token VARCHAR(255) NOT NULL UNIQUE,
 login_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 last_activity TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 logout_time TIMESTAMP NULL,
 is_active BOOLEAN DEFAULT TRUE,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
 FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

Query 12 - Authenticate User

```
SELECT user_id, full_name, email, role, status
FROM users
WHERE (email = 'priya.shah@college.edu' OR phone = '9876500002')
 AND password_hash = SHA2('Priya@123', 256)
 AND status = 'Active'
 AND (locked_until IS NULL OR locked_until < CURRENT_TIMESTAMP);
```

Query 13 - Create Session After Successful Login

```
INSERT INTO login_sessions (user_id, session_token)
SELECT user_id, UUID()
FROM users
WHERE email = 'priya.shah@college.edu'
 AND password_hash = SHA2('Priya@123', 256)
 AND status = 'Active'
 AND (locked_until IS NULL OR locked_until < CURRENT_TIMESTAMP);
```

Query 14 - Reset Failed Attempts After Successful Login

```
UPDATE users
```

```
SET failed_attempts = 0, locked_until = NULL
WHERE user_id = 2;
```

Query 15 - Record Failed Login Attempt

```
UPDATE users
SET failed_attempts = failed_attempts + 1
WHERE email = 'rohan.mehta@college.edu'
 AND status = 'Active';
```

Query 16 - Lock Account After Repeated Failures

```
UPDATE users
SET locked_until = DATE_ADD(CURRENT_TIMESTAMP, INTERVAL 15 MINUTE),
 failed_attempts = 0
WHERE failed_attempts >= 5
 AND status = 'Active';
```

Query 17 - Check Account Status for Pending or Deactivated Login

```
SELECT user_id, full_name, role, status
FROM users
WHERE email = 'karan.joshi@technova.com'
 AND status <> 'Active';
```

Query 18 - View User Login History

```
SELECT session_id, user_id, login_time, logout_time, is_active, created_at,
  updated_at
FROM login_sessions
WHERE user_id = (SELECT user_id FROM users WHERE email =
  'priya.shah@college.edu')
ORDER BY login_time DESC;
```

EP01 - USER MANAGEMENT

TASK T03 - USER LOGOUT

Task ID	T03
User Story ID	EP01-US03
Task Title	User Logout

DATABASE QUERIES

Query 19 - Insert Active Session for Logout Test

```
INSERT INTO login_sessions (user_id, session_token, is_active)
VALUES (3, 'SESSION_ROHAN_01', TRUE);
```

Query 20 - Logout Current User Session

```
UPDATE login_sessions
SET logout_time = CURRENT_TIMESTAMP, is_active = FALSE
WHERE session_token = 'SESSION_ROHAN_01'
 AND is_active = TRUE;
```

Query 21 - End All Active Sessions of a User

```
UPDATE login_sessions
SET logout_time = CURRENT_TIMESTAMP, is_active = FALSE
WHERE user_id = 2
 AND is_active = TRUE;
```

Query 22 - Force Logout Another User (Administrator)

```
UPDATE login_sessions
SET logout_time = CURRENT_TIMESTAMP, is_active = FALSE
WHERE user_id = 3
 AND is_active = TRUE;
```

Query 23 - Automatically End Idle Sessions After 30 Minutes

```
UPDATE login_sessions
SET logout_time = CURRENT_TIMESTAMP, is_active = FALSE
WHERE is_active = TRUE
 AND last_activity < DATE_SUB(CURRENT_TIMESTAMP, INTERVAL 30 MINUTE);
```

Query 24 - Verify Session Status

```
SELECT session_id, user_id, session_token, login_time, last_activity,
  logout_time, is_active,
 created_at, updated_at
FROM login_sessions
```

```
WHERE user_id = 2
ORDER BY login_time DESC;
```

EP01 - USER MANAGEMENT

TASK T04 - MANAGE USER ACCESS

Task ID	T04
User Story ID	EP01-US04
Task Title	Manage User Access

DATABASE QUERIES

Query 25 - Create User Access History Table

```
CREATE TABLE user_access_history (
 history_id INT PRIMARY KEY AUTO_INCREMENT,
 user_id INT NOT NULL,
 action_type VARCHAR(30) NOT NULL,
 old_role VARCHAR(30),
 new_role VARCHAR(30),
 old_status VARCHAR(20),
 new_status VARCHAR(20),
 reason VARCHAR(255),
 action_by INT,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
 FOREIGN KEY (user_id) REFERENCES users(user_id),
 FOREIGN KEY (action_by) REFERENCES users(user_id)
);
```

Query 26 - Insert Test Users for Approval and Role Change

```
INSERT INTO users (full_name, email, phone, password_hash, role, status,
  company_name, designation)
VALUES ('Vikram Rao', 'vikram.rao@quickhire.com', '9876500008',
  SHA2('Vikram@123', 256),
 'Company Recruiter', 'Pending', 'QuickHire Pvt Ltd', 'HR Executive');
INSERT INTO users (full_name, email, phone, password_hash, role, status,
  employee_id, department)
VALUES ('Meera Iyer', 'meera.iyer@college.edu', '9876500009',
  SHA2('Meera@123', 256),
 'Faculty Coordinator', 'Active', 'FC102', 'Information Technology');
```

Query 27 - View Pending Registrations

```
SELECT user_id, full_name, email, role, status, created_at
FROM users
WHERE status = 'Pending'
ORDER BY created_at;
```

Query 28 - Approve Placement Officer Registration (Administrator)

```
INSERT INTO user_access_history (user_id, action_type, old_status, new_status,
  action_by, reason)
VALUES (5, 'APPROVE', 'Pending', 'Active', 1, 'Employee verification
  completed');
UPDATE users SET status = 'Active' WHERE user_id = 5 AND status = 'Pending';
```

Query 29 - Approve Faculty Coordinator Registration (Administrator)

```
INSERT INTO user_access_history (user_id, action_type, old_status, new_status,
  action_by, reason)
VALUES (6, 'APPROVE', 'Pending', 'Active', 1, 'Employee verification
  completed');
UPDATE users SET status = 'Active' WHERE user_id = 6 AND status = 'Pending';
```

Query 30 - Approve Company Recruiter Registration (Placement Officer)

```
INSERT INTO user_access_history (user_id, action_type, old_status, new_status,
  action_by, reason)
SELECT user_id, 'APPROVE', status, 'Active', 5, 'Company verified by Placement
  Officer'
FROM users
WHERE user_id IN (4, 7) AND role = 'Company Recruiter' AND status = 'Pending';
UPDATE users SET status = 'Active'
WHERE user_id IN (4, 7) AND role = 'Company Recruiter' AND status = 'Pending';
```

Query 31 - Reject Pending Registration

```
INSERT INTO user_access_history (user_id, action_type, old_status, new_status,
  action_by, reason)
VALUES (8, 'REJECT', 'Pending', 'Rejected', 5, 'Company could not be
  verified');
UPDATE users SET status = 'Rejected' WHERE user_id = 8 AND status = 'Pending';
```

Query 32 - Deactivate User Account

```
INSERT INTO user_access_history (user_id, action_type, old_status, new_status,
  action_by, reason)
SELECT user_id, 'DEACTIVATE', status, 'Deactivated', 1, 'Account access
  suspended'
FROM users WHERE user_id = 9;
UPDATE users SET status = 'Deactivated' WHERE user_id = 9;
```

Query 33 - Reactivate User Account

```
INSERT INTO user_access_history (user_id, action_type, old_status, new_status,
  action_by, reason)
```

```
SELECT user_id, 'REACTIVATE', status, 'Active', 1, 'Suspension period
  completed'
FROM users WHERE user_id = 9 AND status = 'Deactivated';
UPDATE users SET status = 'Active' WHERE user_id = 9 AND status =
  'Deactivated';
```

Query 34 - Change User Role

```
INSERT INTO user_access_history (user_id, action_type, old_role, new_role,
  action_by, reason)
SELECT user_id, 'ROLE_CHANGE', role, 'Placement Officer', 1, 'Role updated by
  Administrator'
FROM users WHERE user_id = 9;
UPDATE users SET role = 'Placement Officer' WHERE user_id = 9;
```

Query 35 - Delete User Account (Soft Delete)

```
INSERT INTO user_access_history (user_id, action_type, old_role, old_status,
  new_status, action_by, reason)
SELECT user_id, 'DELETE', role, status, 'Deleted', 1, 'Account removed on
  request'
FROM users WHERE user_id = 9;
UPDATE users SET status = 'Deleted' WHERE user_id = 9;
```

Query 36 - View Access History

```
SELECT history_id, user_id, action_type, old_role, new_role, old_status,
  new_status, reason,
 action_by, created_at, updated_at
FROM user_access_history
ORDER BY created_at DESC;
```

EP02 - STUDENT PROFILE MANAGEMENT

TASK T05 - CREATE STUDENT PROFILE

Task ID	T05
User Story ID	EP02-US01
Task Title	Create Student Profile

DATABASE QUERIES

Query 37 - Create Student Profile Table

```
CREATE TABLE student_profiles (
 student_id INT PRIMARY KEY AUTO_INCREMENT,
 user_id INT NOT NULL UNIQUE,
 enrollment_no VARCHAR(30) NOT NULL UNIQUE,
 branch VARCHAR(50) NOT NULL,
 semester INT NOT NULL,
 cgpa DECIMAL(4,2) NOT NULL,
 backlogs INT DEFAULT 0,
 skills VARCHAR(255),
 address VARCHAR(255),
 date_of_birth DATE,
 linkedin_url VARCHAR(150),
 github_url VARCHAR(150),
 verification_status VARCHAR(30) DEFAULT 'Pending',
 verification_remarks VARCHAR(255),
 verified_by INT,
 verified_at TIMESTAMP NULL,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
 FOREIGN KEY (user_id) REFERENCES users(user_id),
 FOREIGN KEY (verified_by) REFERENCES users(user_id)
);
```

Query 38 - Insert Student Profile - Priya Shah

```
INSERT INTO student_profiles (user_id, enrollment_no, branch, semester, cgpa,
  backlogs, skills,
 address, date_of_birth)
SELECT user_id, enrollment_no, branch, semester, 8.60, 0, 'Java, SQL, React',
 address, '2005-03-14'
FROM users
WHERE user_id = 2 AND role = 'Student' AND status = 'Active';
```

Query 39 - Insert Student Profile - Rohan Mehta

```
INSERT INTO student_profiles (user_id, enrollment_no, branch, semester, cgpa,
  backlogs, skills,
 address, date_of_birth)
SELECT user_id, enrollment_no, branch, semester, 6.90, 2, 'HTML, CSS,
  JavaScript',
 address, '2005-07-21'
FROM users
WHERE user_id = 3 AND role = 'Student' AND status = 'Active';
```

Query 40 - Check Existing Profile or Duplicate Enrollment Number

```
SELECT student_id, user_id, enrollment_no
FROM student_profiles
WHERE user_id = 2
 OR enrollment_no = 'DCSE2023002';
```

Query 41 - Find Profiles with Invalid Academic Values

```
SELECT student_id, enrollment_no, cgpa, backlogs, semester
FROM student_profiles
WHERE cgpa NOT BETWEEN 0 AND 10
 OR backlogs < 0
 OR semester NOT BETWEEN 1 AND 6;
```

Query 42 - View Profiles Pending Verification

```
SELECT s.student_id, u.full_name, s.enrollment_no, s.branch, s.semester,
  s.cgpa, s.backlogs,
 s.verification_status
FROM student_profiles s
JOIN users u ON s.user_id = u.user_id
WHERE s.verification_status = 'Pending'
ORDER BY s.created_at;
```

Query 43 - Verify Academic Details (Placement Officer)

```
UPDATE student_profiles
SET verification_status = 'Verified', verification_remarks = 'Marksheet
  checked',
 verified_by = 5, verified_at = CURRENT_TIMESTAMP
WHERE student_id = 1 AND verification_status = 'Pending';
```

Query 44 - Request Correction of Academic Details (Placement Officer)

```
UPDATE student_profiles
SET verification_status = 'Correction Requested',
 verification_remarks = 'CGPA does not match the latest marksheet',
```

```
 verified_by = 5, verified_at = CURRENT_TIMESTAMP
WHERE student_id = 2 AND verification_status = 'Pending';
```

EP02 - STUDENT PROFILE MANAGEMENT

TASK T06 - VIEW STUDENT PROFILE

Task ID	T06
User Story ID	EP02-US02
Task Title	View Student Profile

DATABASE QUERIES

Query 45 - Student Views Own Profile

```
SELECT s.student_id, u.full_name, u.email, u.phone, s.enrollment_no, s.branch,
  s.semester, s.cgpa,
 s.backlogs, s.skills, s.address, s.verification_status, s.created_at,
   s.updated_at
FROM student_profiles s
JOIN users u ON s.user_id = u.user_id
WHERE s.user_id = 2;
```

Query 46 - Placement Officer Searches Student Profile

```
SELECT s.student_id, u.full_name, u.email, u.phone, s.enrollment_no, s.branch,
  s.semester,
 s.cgpa, s.backlogs, s.verification_status
FROM student_profiles s
JOIN users u ON s.user_id = u.user_id
WHERE u.full_name LIKE '%Priya%'
 OR s.enrollment_no = 'DCSE2023002'
 OR s.branch = 'CSE'
ORDER BY u.full_name;
```

Query 47 - Faculty Coordinator Views Department Students (Masked Contact)

```
SELECT s.student_id, u.full_name, s.enrollment_no, s.semester, s.cgpa,
  s.verification_status,
 CONCAT('******', RIGHT(u.phone, 4)) AS masked_phone
FROM student_profiles s
JOIN users u ON s.user_id = u.user_id
WHERE s.branch = 'CSE'
ORDER BY s.enrollment_no;
```

Query 48 - Administrator Views Complete Student Records

```
SELECT s.student_id, s.user_id, u.full_name, u.email, u.phone, u.status AS
  account_status,
 s.enrollment_no, s.branch, s.semester, s.cgpa, s.backlogs, s.skills,
   s.verification_status,
```

```
 s.created_at, s.updated_at
FROM student_profiles s
JOIN users u ON s.user_id = u.user_id
ORDER BY s.enrollment_no;
```

Query 49 - Company Recruiter Views Applicant Profile (run after Task T13)

```
SELECT s.student_id, u.full_name, s.branch, s.semester, s.cgpa, s.backlogs,
  s.skills,
 r.file_name AS resume_file
FROM applications a
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
JOIN student_profiles s ON a.student_id = s.student_id
JOIN users u ON s.user_id = u.user_id
LEFT JOIN student_resumes r ON a.resume_id = r.resume_id
WHERE c.user_id = 4
 AND o.opportunity_id = 1;
```

EP02 - STUDENT PROFILE MANAGEMENT

TASK T07 - EDIT STUDENT PROFILE

Task ID	T07
User Story ID	EP02-US03
Task Title	Edit Student Profile

DATABASE QUERIES

Query 50 - Create Student Edit History Table

```
CREATE TABLE student_edit_history (
 history_id INT PRIMARY KEY AUTO_INCREMENT,
 student_id INT NOT NULL,
 field_name VARCHAR(100) NOT NULL,
 old_value VARCHAR(255),
 new_value VARCHAR(255),
 reason VARCHAR(255),
 edited_by INT NOT NULL,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
 FOREIGN KEY (student_id) REFERENCES student_profiles(student_id),
 FOREIGN KEY (edited_by) REFERENCES users(user_id)
);
```

Query 51 - Student Updates Skills and Address

```
INSERT INTO student_edit_history (student_id, field_name, old_value,
  new_value, reason, edited_by)
SELECT student_id, 'skills', skills, 'Java, SQL, React, Spring Boot', 'Learned
  new skill', 2
FROM student_profiles WHERE student_id = 1;
UPDATE student_profiles
SET skills = 'Java, SQL, React, Spring Boot'
WHERE student_id = 1 AND user_id = 2;
```

Query 52 - Student Changes CGPA (Re-verification Required)

```
INSERT INTO student_edit_history (student_id, field_name, old_value,
  new_value, reason, edited_by)
SELECT student_id, 'cgpa', cgpa, '7.10', 'Corrected as per latest marksheet',
  3
FROM student_profiles WHERE student_id = 2;
UPDATE student_profiles
SET cgpa = 7.10, verification_status = 'Pending', verification_remarks = NULL
WHERE student_id = 2 AND user_id = 3;
```

Query 53 - Placement Officer Corrects and Verifies Academic Details

```
INSERT INTO student_edit_history (student_id, field_name, old_value,
  new_value, reason, edited_by)
SELECT student_id, 'backlogs', backlogs, '1', 'Backlog count verified from
  result sheet', 5
FROM student_profiles WHERE student_id = 2;
UPDATE student_profiles
SET backlogs = 1, verification_status = 'Verified', verification_remarks =
  'Corrected and verified',
 verified_by = 5, verified_at = CURRENT_TIMESTAMP
WHERE student_id = 2;
```

Query 54 - View Edit History of a Student

```
SELECT history_id, student_id, field_name, old_value, new_value, reason,
  edited_by, created_at,
 updated_at
FROM student_edit_history
WHERE student_id = 1
ORDER BY created_at DESC;
```

Query 55 - Faculty Coordinator Views Department Edit History (Read-Only)

```
SELECT h.history_id, u.full_name, h.field_name, h.old_value, h.new_value,
  h.reason, h.created_at
FROM student_edit_history h
JOIN student_profiles s ON h.student_id = s.student_id
JOIN users u ON s.user_id = u.user_id
WHERE s.branch = 'CSE'
ORDER BY h.created_at DESC;
```

EP02 - STUDENT PROFILE MANAGEMENT

TASK T08 - UPLOAD AND MANAGE RESUME

Task ID	T08
User Story ID	EP02-US04
Task Title	Upload and Manage Resume

DATABASE QUERIES

Query 56 - Create Student Resume Table

```
CREATE TABLE student_resumes (
 resume_id INT PRIMARY KEY AUTO_INCREMENT,
 student_id INT NOT NULL,
 file_name VARCHAR(150) NOT NULL,
 file_path VARCHAR(255) NOT NULL,
 file_type VARCHAR(10) NOT NULL,
 file_size_kb INT NOT NULL,
 is_current BOOLEAN DEFAULT TRUE,
 removal_reason VARCHAR(255),
 removed_by INT,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
 FOREIGN KEY (student_id) REFERENCES student_profiles(student_id),
 FOREIGN KEY (removed_by) REFERENCES users(user_id)
);
```

Query 57 - Upload Resume - Priya Shah

```
INSERT INTO student_resumes (student_id, file_name, file_path, file_type,
  file_size_kb)
VALUES (1, 'Priya_Shah_Resume_v1.pdf', '/uploads/resumes/priya_shah_v1.pdf',
  'PDF', 410);
```

Query 58 - Replace Existing Resume

```
UPDATE student_resumes SET is_current = FALSE WHERE student_id = 1 AND
  is_current = TRUE;
INSERT INTO student_resumes (student_id, file_name, file_path, file_type,
  file_size_kb)
VALUES (1, 'Priya_Shah_Resume_v2.pdf', '/uploads/resumes/priya_shah_v2.pdf',
  'PDF', 455);
```

Query 59 - Upload Resume - Rohan Mehta

```
INSERT INTO student_resumes (student_id, file_name, file_path, file_type,
```

```
  file_size_kb)
VALUES (2, 'Rohan_Mehta_Resume.pdf', '/uploads/resumes/rohan_mehta_v1.pdf',
  'PDF', 380);
```

Query 60 - View Current Resume of a Student

```
SELECT resume_id, student_id, file_name, file_path, file_type, file_size_kb,
  created_at, updated_at
FROM student_resumes
WHERE student_id = 1 AND is_current = TRUE;
```

Query 61 - Student Deletes Own Resume Version

```
UPDATE student_resumes
SET is_current = FALSE, removal_reason = 'Deleted by student', removed_by = 2
WHERE resume_id = 1 AND student_id = 1;
```

Query 62 - Placement Officer Removes Inappropriate Resume

```
UPDATE student_resumes
SET is_current = FALSE, removal_reason = 'Resume contains inappropriate
  content', removed_by = 5
WHERE resume_id = 1 AND removed_by IS NULL;
```

Query 63 - Find Resumes Violating File Rules

```
SELECT resume_id, student_id, file_name, file_type, file_size_kb
FROM student_resumes
WHERE file_type <> 'PDF'
 OR file_size_kb > 2048;
```

EP03 - COMPANY AND OPPORTUNITY MANAGEMENT

TASK T09 - CREATE COMPANY PROFILE

Task ID	T09
User Story ID	EP03-US01
Task Title	Create Company Profile

DATABASE QUERIES

Query 64 - Create Company Table

```
CREATE TABLE companies (
 company_id INT PRIMARY KEY AUTO_INCREMENT,
 user_id INT NOT NULL UNIQUE,
 company_name VARCHAR(150) NOT NULL,
 industry VARCHAR(100),
 website VARCHAR(150),
 location VARCHAR(100),
 hr_name VARCHAR(100),
 hr_email VARCHAR(100),
 hr_phone VARCHAR(15),
 description TEXT,
 company_size VARCHAR(30),
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
 FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

Query 65 - Insert Company Profile - TechNova Solutions

```
INSERT INTO companies (user_id, company_name, industry, website, location,
  hr_name, hr_email, hr_phone,
 description, company_size)
SELECT user_id, 'TechNova Solutions', 'IT Services', 'www.technova.com',
  'Bengaluru', full_name, email,
 phone, 'Software services and product engineering company', '201-500'
FROM users
WHERE user_id = 4 AND role = 'Company Recruiter' AND status = 'Active';
```

Query 66 - Insert Company Profile - BrightPath Infotech

```
INSERT INTO companies (user_id, company_name, industry, website, location,
  hr_name, hr_email, hr_phone,
 description, company_size)
SELECT user_id, 'BrightPath Infotech', 'Software Product',
  'www.brightpath.in', 'Hyderabad', full_name,
```

```
 email, phone, 'Web and mobile product development company', '51-200'
FROM users
WHERE user_id = 7 AND role = 'Company Recruiter' AND status = 'Active';
```

Query 67 - Check Duplicate Company Name or Website

```
SELECT company_id, company_name, website
FROM companies
WHERE company_name = 'TechNova Solutions'
 OR website = 'www.technova.com';
```

Query 68 - Validate Mandatory Company Fields

```
SELECT company_id, company_name, hr_email, hr_phone
FROM companies
WHERE company_name IS NULL OR company_name = ''
 OR hr_email IS NULL OR hr_email NOT LIKE '%@%.%';
```

Query 69 - Update Company Profile

```
UPDATE companies
SET hr_phone = '9876500044', company_size = '501-1000'
WHERE company_id = 1 AND user_id = 4;
```

Query 70 - View All Companies with Recruiter Account Status

```
SELECT c.company_id, c.company_name, c.industry, c.location, c.hr_name,
  c.hr_email, u.status AS recruiter_status,
 c.created_at, c.updated_at
FROM companies c
JOIN users u ON c.user_id = u.user_id
ORDER BY c.company_name;
```

EP03 - COMPANY AND OPPORTUNITY MANAGEMENT

TASK T10 - POST OPPORTUNITY

Task ID	T10
User Story ID	EP03-US02
Task Title	Post Opportunity

DATABASE QUERIES

Query 71 - Create Opportunity Table

```
CREATE TABLE opportunities (
 opportunity_id INT PRIMARY KEY AUTO_INCREMENT,
 company_id INT NOT NULL,
 title VARCHAR(150) NOT NULL,
 type VARCHAR(20) NOT NULL,
 description TEXT,
 location VARCHAR(100),
 min_cgpa DECIMAL(4,2) DEFAULT 0,
 eligible_branches VARCHAR(100),
 min_semester INT DEFAULT 1,
 max_backlogs INT DEFAULT 0,
 compensation DECIMAL(10,2),
 vacancies INT DEFAULT 1,
 application_deadline DATE NOT NULL,
 status VARCHAR(20) DEFAULT 'Open',
 posted_by INT NOT NULL,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
 FOREIGN KEY (company_id) REFERENCES companies(company_id),
 FOREIGN KEY (posted_by) REFERENCES users(user_id)
);
```

Query 72 - Post Internship - Java Developer Intern

```
INSERT INTO opportunities (company_id, title, type, description, location,
  min_cgpa, eligible_branches,
 min_semester, max_backlogs, compensation, vacancies, application_deadline,
   posted_by)
VALUES (1, 'Java Developer Intern', 'Internship',
 'Work on Spring Boot REST APIs and MySQL under mentor guidance', 'Bengaluru',
   7.00, 'CSE,IT', 5, 0,
 15000, 5, '2026-11-30', 4);
```

Query 73 - Post Placement - Software Engineer Trainee

```
INSERT INTO opportunities (company_id, title, type, description, location,
  min_cgpa, eligible_branches,
 min_semester, max_backlogs, compensation, vacancies, application_deadline,
   posted_by)
VALUES (1, 'Software Engineer Trainee', 'Placement',
 'Full-time trainee role in software development with 6 months training',
   'Bengaluru', 7.50, 'CSE,IT',
 5, 0, 350000, 10, '2026-12-15', 4);
```

Query 74 - Post Internship - Web Developer Intern

```
INSERT INTO opportunities (company_id, title, type, description, location,
  min_cgpa, eligible_branches,
 min_semester, max_backlogs, compensation, vacancies, application_deadline,
   posted_by)
VALUES (2, 'Web Developer Intern', 'Internship',
 'Build responsive web pages using React and REST APIs', 'Hyderabad', 6.00,
   'CSE,IT', 4, 2, 10000, 3,
'2026-11-20', 7);
```

Query 75 - Placement Officer Posts Opportunity on Behalf of Company

```
INSERT INTO opportunities (company_id, title, type, description, location,
  min_cgpa, eligible_branches,
 min_semester, max_backlogs, compensation, vacancies, application_deadline,
   posted_by)
VALUES (2, 'Associate Developer', 'Placement',
 'Campus drive arranged offline; role in product development team',
   'Hyderabad', 6.50, 'CSE', 5, 0,
 450000, 4, '2026-12-20', 5);
```

Query 76 - Find Invalid Opportunity Data

```
SELECT opportunity_id, title, min_cgpa, application_deadline
FROM opportunities
WHERE title IS NULL OR title = ''
 OR min_cgpa NOT BETWEEN 0 AND 10
 OR application_deadline < CURDATE();
```

Query 77 - Automatically Close Opportunities After Deadline

```
UPDATE opportunities
SET status = 'Closed'
WHERE application_deadline < CURDATE()
 AND status = 'Open';
```

Query 78 - View Open Opportunities with Company

```
SELECT o.opportunity_id, o.title, c.company_name, o.type, o.location,
  o.min_cgpa, o.compensation,
 o.vacancies, o.application_deadline, o.status
FROM opportunities o
JOIN companies c ON o.company_id = c.company_id
WHERE o.status = 'Open'
ORDER BY o.application_deadline;
```

EP03 - COMPANY AND OPPORTUNITY MANAGEMENT

TASK T11 - EDIT OPPORTUNITY

Task ID	T11
User Story ID	EP03-US03
Task Title	Edit Opportunity

DATABASE QUERIES

Query 79 - Create Opportunity Edit History Table

```
CREATE TABLE opportunity_edit_history (
 history_id INT PRIMARY KEY AUTO_INCREMENT,
 opportunity_id INT NOT NULL,
 field_name VARCHAR(100) NOT NULL,
 old_value VARCHAR(255),
 new_value VARCHAR(255),
 reason VARCHAR(255),
 edited_by INT NOT NULL,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
 FOREIGN KEY (opportunity_id) REFERENCES opportunities(opportunity_id),
 FOREIGN KEY (edited_by) REFERENCES users(user_id)
);
```

Query 80 - Recruiter Edits Own Opportunity

```
INSERT INTO opportunity_edit_history (opportunity_id, field_name, old_value,
  new_value, reason, edited_by)
SELECT o.opportunity_id, 'compensation', o.compensation, '18000', 'Stipend
  revised by company', 4
FROM opportunities o
JOIN companies c ON o.company_id = c.company_id
WHERE o.opportunity_id = 1 AND c.user_id = 4;
UPDATE opportunities o
JOIN companies c ON o.company_id = c.company_id
SET o.compensation = 18000
WHERE o.opportunity_id = 1 AND c.user_id = 4;
```

Query 81 - Placement Officer Extends Deadline

```
INSERT INTO opportunity_edit_history (opportunity_id, field_name, old_value,
  new_value, reason, edited_by)
SELECT opportunity_id, 'application_deadline', CAST(application_deadline AS
  CHAR), '2026-11-25',
 'Extended on company request', 5
```

```
FROM opportunities WHERE opportunity_id = 3;
UPDATE opportunities SET application_deadline = '2026-11-25' WHERE
  opportunity_id = 3;
```

Query 82 - Administrator Corrects Opportunity Location

```
INSERT INTO opportunity_edit_history (opportunity_id, field_name, old_value,
  new_value, reason, edited_by)
SELECT opportunity_id, 'location', location, 'Bengaluru (Hybrid)', 'Location
  clarified', 1
FROM opportunities WHERE opportunity_id = 2;
UPDATE opportunities SET location = 'Bengaluru (Hybrid)' WHERE opportunity_id
  = 2;
```

Query 83 - View Edit History of an Opportunity

```
SELECT history_id, opportunity_id, field_name, old_value, new_value, reason,
  edited_by, created_at,
 updated_at
FROM opportunity_edit_history
WHERE opportunity_id = 1
ORDER BY created_at DESC;
```

Query 84 - Administrator Views All Recent Opportunity Edits

```
SELECT h.history_id, o.title, c.company_name, h.field_name, h.old_value,
  h.new_value, h.reason,
 u.full_name AS edited_by, h.created_at
FROM opportunity_edit_history h
JOIN opportunities o ON h.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
JOIN users u ON h.edited_by = u.user_id
ORDER BY h.created_at DESC;
```

EP03 - COMPANY AND OPPORTUNITY MANAGEMENT

TASK T12 - CLOSE OR DELETE OPPORTUNITY

Task ID	T12
User Story ID	EP03-US04
Task Title	Close or Delete Opportunity

DATABASE QUERIES

Query 85 - Create Opportunity Deletion History Table

```
CREATE TABLE opportunity_deletion_history (
 deletion_id INT PRIMARY KEY AUTO_INCREMENT,
 opportunity_id INT NOT NULL,
 title VARCHAR(150) NOT NULL,
 previous_status VARCHAR(20),
 reason VARCHAR(255) NOT NULL,
 deleted_by INT NOT NULL,
 deleted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 restored_at TIMESTAMP NULL,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
 FOREIGN KEY (opportunity_id) REFERENCES opportunities(opportunity_id),
 FOREIGN KEY (deleted_by) REFERENCES users(user_id)
);
```

Query 86 - Insert Test Opportunity for Close and Delete Demonstration

```
INSERT INTO opportunities (company_id, title, type, location, min_cgpa,
  eligible_branches, compensation,
 vacancies, application_deadline, posted_by)
VALUES (2, 'Test Posting - Duplicate Entry', 'Internship', 'Hyderabad', 6.00,
  'CSE', 8000, 1,
 '2026-12-31', 7);
```

Query 87 - Recruiter Closes Own Opportunity

```
UPDATE opportunities o
JOIN companies c ON o.company_id = c.company_id
SET o.status = 'Closed'
WHERE o.opportunity_id = 5 AND c.user_id = 7 AND o.status = 'Open';
```

Query 88 - Check Selected Applicants Before Deletion (run after Task T13)

```
SELECT COUNT(*) AS selected_applicants
FROM applications
```

```
WHERE opportunity_id = 5
 AND status = 'Selected';
```

Query 89 - Delete Opportunity (Soft Delete)

```
INSERT INTO opportunity_deletion_history (opportunity_id, title,
  previous_status, reason, deleted_by)
SELECT opportunity_id, title, status, 'Posted by mistake', 5
FROM opportunities
WHERE opportunity_id = 5;
UPDATE opportunities SET status = 'Deleted' WHERE opportunity_id = 5;
```

Query 90 - View Deletion History

```
SELECT deletion_id, opportunity_id, title, previous_status, reason,
  deleted_by, deleted_at, restored_at,
 created_at, updated_at
FROM opportunity_deletion_history
ORDER BY deleted_at DESC;
```

Query 91 - Restore Deleted Opportunity

```
UPDATE opportunities o
JOIN opportunity_deletion_history h ON o.opportunity_id = h.opportunity_id
SET o.status = h.previous_status, h.restored_at = CURRENT_TIMESTAMP
WHERE o.opportunity_id = 5
 AND o.status = 'Deleted'
 AND h.restored_at IS NULL;
```

Query 92 - Faculty Coordinator Views Closed and Deleted Opportunities (Read-Only)

```
SELECT o.opportunity_id, o.title, c.company_name, o.type, o.status,
  o.application_deadline
FROM opportunities o
JOIN companies c ON o.company_id = c.company_id
WHERE o.status IN ('Closed', 'Deleted')
ORDER BY o.application_deadline DESC;
```

EP04 - APPLICATION MANAGEMENT

TASK T13 - APPLY FOR OPPORTUNITY

Task ID	T13
User Story ID	EP04-US01
Task Title	Apply for Opportunity

DATABASE QUERIES

Query 93 - Create Application Table

```
CREATE TABLE applications (
 application_id INT PRIMARY KEY AUTO_INCREMENT,
 student_id INT NOT NULL,
 opportunity_id INT NOT NULL,
 resume_id INT,
 applied_on DATE NOT NULL,
 status VARCHAR(30) DEFAULT 'Applied',
 applied_by INT,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
 UNIQUE (student_id, opportunity_id),
 FOREIGN KEY (student_id) REFERENCES student_profiles(student_id),
 FOREIGN KEY (opportunity_id) REFERENCES opportunities(opportunity_id),
 FOREIGN KEY (resume_id) REFERENCES student_resumes(resume_id),
 FOREIGN KEY (applied_by) REFERENCES users(user_id)
);
```

Query 94 - Create Application Status History Table

```
CREATE TABLE application_status_history (
 history_id INT PRIMARY KEY AUTO_INCREMENT,
 application_id INT NOT NULL,
 old_status VARCHAR(30),
 new_status VARCHAR(30) NOT NULL,
 changed_by INT NOT NULL,
 change_reason VARCHAR(255),
 changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
 FOREIGN KEY (application_id) REFERENCES applications(application_id),
 FOREIGN KEY (changed_by) REFERENCES users(user_id)
);
```

Query 95 - List Open Opportunities Eligible for a Student

```
SELECT o.opportunity_id, o.title, o.type, o.min_cgpa, o.application_deadline
FROM opportunities o
JOIN student_profiles s ON s.student_id = 1
WHERE o.status = 'Open'
 AND o.application_deadline >= CURDATE()
 AND s.verification_status = 'Verified'
 AND s.cgpa >= o.min_cgpa
  AND s.backlogs <= o.max_backlogs
  AND s.semester >= o.min_semester
  AND FIND_IN_SET(s.branch, o.eligible_branches) > 0
ORDER BY o.application_deadline;
```

Query 96 - Show Eligibility Result with Reason

```
SELECT o.title,
 CASE
 WHEN o.status <> 'Open' THEN 'Opportunity is closed'
 WHEN o.application_deadline < CURDATE() THEN 'Application deadline is over'
 WHEN s.cgpa < o.min_cgpa THEN CONCAT('Minimum CGPA required is ', o.min_cgpa)
 WHEN s.backlogs > o.max_backlogs THEN CONCAT('Maximum backlogs allowed is ',
   o.max_backlogs)
 WHEN s.semester < o.min_semester THEN 'Semester requirement not met'
 WHEN FIND_IN_SET(s.branch, o.eligible_branches) = 0 THEN 'Branch is not
   eligible'
 ELSE 'Eligible'
 END AS eligibility_result
FROM student_profiles s
JOIN opportunities o ON o.opportunity_id = 1
WHERE s.student_id = 2;
```

Query 97 - Apply for Opportunity with Eligibility and Duplicate Check

```
INSERT INTO applications (student_id, opportunity_id, resume_id, applied_on,
  status)
SELECT s.student_id, o.opportunity_id,
 (SELECT r.resume_id FROM student_resumes r
 WHERE r.student_id = s.student_id AND r.is_current = TRUE LIMIT 1),
 CURDATE(), 'Applied'
FROM student_profiles s
JOIN opportunities o ON o.opportunity_id = 1
WHERE s.student_id = 1
 AND o.status = 'Open'
 AND o.application_deadline >= CURDATE()
 AND s.cgpa >= o.min_cgpa
  AND s.backlogs <= o.max_backlogs
  AND s.semester >= o.min_semester
```

```
 AND FIND_IN_SET(s.branch, o.eligible_branches) > 0
AND NOT EXISTS (SELECT 1 FROM applications a
WHERE a.student_id = s.student_id AND a.opportunity_id = o.opportunity_id);
```

Query 98 - Insert Further Applications

```
INSERT INTO applications (student_id, opportunity_id, resume_id, applied_on,
 status)
VALUES (1, 2, 2, CURDATE(), 'Applied'),
(2, 3, 3, CURDATE(), 'Applied');
```

Query 99 - Placement Officer Applies on Behalf of a Student

```
INSERT INTO applications (student_id, opportunity_id, resume_id, applied_on,
 status, applied_by)
VALUES (1, 4, 2, CURDATE(), 'Applied', 5);
```

Query 100 - Insert Initial Status History for New Applications

```
INSERT INTO application_status_history (application_id, old_status,
 new_status, changed_by, change_reason)
SELECT a.application_id, NULL, a.status, COALESCE(a.applied_by, s.user_id),
 'Application submitted'
FROM applications a
JOIN student_profiles s ON a.student_id = s.student_id
WHERE NOT EXISTS (SELECT 1 FROM application_status_history h
 WHERE h.application_id = a.application_id);
```

Query 101 - Check Duplicate Application

```
SELECT application_id, student_id, opportunity_id, status
FROM applications
WHERE student_id = 1
 AND opportunity_id = 1;
```

Query 102 - Check Placement Restriction After Accepted Offer (run after Task T20)

```
SELECT a.student_id, o.title, f.status AS offer_status
FROM offers f
JOIN applications a ON f.application_id = a.application_id
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
WHERE a.student_id = 1
 AND o.type = 'Placement'
 AND f.status = 'Accepted';
```

EP04 - APPLICATION MANAGEMENT

TASK T14 - VIEW APPLICATION DETAILS

Task ID	T14
User Story ID	EP04-US02
Task Title	View Application Details

DATABASE QUERIES

Query 103 - Student Views Own Applications

```
SELECT a.application_id, o.title, c.company_name, o.type, a.applied_on,
  a.status, a.created_at,
 a.updated_at
FROM applications a
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
WHERE a.student_id = 1
ORDER BY a.applied_on DESC;
```

Query 104 - Company Recruiter Views Applications for Own Opportunities

```
SELECT a.application_id, o.title, u.full_name, s.branch, s.cgpa, s.skills,
  r.file_name AS resume,
 a.applied_on, a.status
FROM applications a
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
JOIN student_profiles s ON a.student_id = s.student_id
JOIN users u ON s.user_id = u.user_id
LEFT JOIN student_resumes r ON a.resume_id = r.resume_id
WHERE c.user_id = 4
ORDER BY o.title, a.applied_on;
```

Query 105 - Placement Officer Views All Applications

```
SELECT a.application_id, u.full_name AS student, s.branch, c.company_name,
  o.title, a.applied_on,
 a.status
FROM applications a
JOIN student_profiles s ON a.student_id = s.student_id
JOIN users u ON s.user_id = u.user_id
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
ORDER BY a.applied_on DESC;
```

Query 106 - Faculty Coordinator Views Department Application Summary (Read-Only)

```
SELECT u.full_name, a.status, COUNT(*) AS total_applications
FROM applications a
JOIN student_profiles s ON a.student_id = s.student_id
JOIN users u ON s.user_id = u.user_id
WHERE s.branch = 'CSE'
GROUP BY u.full_name, a.status
ORDER BY u.full_name, a.status;
```

Query 107 - Administrator Views Complete Application Details

```
SELECT a.application_id, a.student_id, a.opportunity_id, a.resume_id,
  a.applied_on, a.status,
 a.applied_by, a.created_at, a.updated_at
FROM applications a
ORDER BY a.application_id;
```

Query 108 - View Application Status History

```
SELECT history_id, application_id, old_status, new_status, changed_by,
  change_reason, changed_at,
 created_at, updated_at
FROM application_status_history
WHERE application_id = 1
ORDER BY changed_at;
```

EP04 - APPLICATION MANAGEMENT

TASK T15 - UPDATE APPLICATION STATUS

Task ID	T15
User Story ID	EP04-US03
Task Title	Update Application Status

DATABASE QUERIES

Query 109 - Recruiter Shortlists Applicant - TechNova

```
INSERT INTO application_status_history (application_id, old_status,
  new_status, changed_by, change_reason)
SELECT a.application_id, a.status, 'Shortlisted', 4, 'Profile matches
  requirement'
FROM applications a
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
WHERE a.application_id = 1 AND c.user_id = 4 AND a.status = 'Applied';
UPDATE applications a
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
SET a.status = 'Shortlisted'
WHERE a.application_id = 1 AND c.user_id = 4 AND a.status = 'Applied';
```

Query 110 - Recruiter Shortlists Applicant - BrightPath

```
INSERT INTO application_status_history (application_id, old_status,
  new_status, changed_by, change_reason)
SELECT a.application_id, a.status, 'Shortlisted', 7, 'Skills match the role'
FROM applications a
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
WHERE a.application_id = 3 AND c.user_id = 7 AND a.status = 'Applied';
UPDATE applications a
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
SET a.status = 'Shortlisted'
WHERE a.application_id = 3 AND c.user_id = 7 AND a.status = 'Applied';
```

Query 111 - Validate Allowed Status Transition Before Selection

```
SELECT application_id, status
FROM applications
WHERE application_id = 1
 AND status IN ('Shortlisted', 'Interview Scheduled');
```

Query 112 - Placement Officer Corrects Application Status

```
INSERT INTO application_status_history (application_id, old_status,
  new_status, changed_by, change_reason)
SELECT application_id, status, 'Shortlisted', 5, 'Corrected after eligibility
  review'
FROM applications
WHERE application_id = 4 AND status IN ('Applied');
UPDATE applications SET status = 'Shortlisted' WHERE application_id = 4 AND
  status = 'Applied';
```

Query 113 - Recruiter Rejects Application with Reason

```
INSERT INTO application_status_history (application_id, old_status,
  new_status, changed_by, change_reason)
SELECT application_id, status, 'Rejected', 4, 'Vacancies filled'
FROM applications
WHERE application_id = 2 AND status IN ('Applied', 'Shortlisted', 'Interview
  Scheduled');
UPDATE applications SET status = 'Rejected'
WHERE application_id = 2 AND status IN ('Applied', 'Shortlisted', 'Interview
  Scheduled');
```

Query 114 - Recruiter Marks Applicant as Selected - TechNova (after interview)

```
INSERT INTO application_status_history (application_id, old_status,
  new_status, changed_by, change_reason)
SELECT application_id, status, 'Selected', 4, 'Selected after final interview'
FROM applications
WHERE application_id = 1 AND status IN ('Shortlisted', 'Interview Scheduled');
UPDATE applications SET status = 'Selected'
WHERE application_id = 1 AND status IN ('Shortlisted', 'Interview Scheduled');
```

Query 115 - Recruiter Marks Applicant as Selected - BrightPath (after interview)

```
INSERT INTO application_status_history (application_id, old_status,
  new_status, changed_by, change_reason)
SELECT application_id, status, 'Selected', 7, 'Selected after technical round'
FROM applications
WHERE application_id = 3 AND status IN ('Shortlisted', 'Interview Scheduled');
UPDATE applications SET status = 'Selected'
WHERE application_id = 3 AND status IN ('Shortlisted', 'Interview Scheduled');
```

Query 116 - View Complete Status Update History

```
SELECT h.history_id, h.application_id, h.old_status, h.new_status, u.full_name
  AS changed_by,
 h.change_reason, h.changed_at
FROM application_status_history h
```

```
JOIN users u ON h.changed_by = u.user_id
ORDER BY h.application_id, h.changed_at;
```

EP04 - APPLICATION MANAGEMENT

TASK T16 - WITHDRAW APPLICATION

Task ID	T16
User Story ID	EP04-US04
Task Title	Withdraw Application

DATABASE QUERIES

Query 117 - Student Withdraws Application

```
INSERT INTO application_status_history (application_id, old_status,
  new_status, changed_by, change_reason)
SELECT application_id, status, 'Withdrawn', 2, 'Student chose another
  opportunity'
FROM applications
WHERE application_id = 4 AND student_id = 1
 AND status NOT IN ('Selected', 'Rejected', 'Withdrawn');
UPDATE applications SET status = 'Withdrawn'
WHERE application_id = 4 AND student_id = 1
 AND status NOT IN ('Selected', 'Rejected', 'Withdrawn');
```

Query 118 - Block Withdrawal When an Offer Is Accepted (run after Task T20)

```
SELECT a.application_id, f.status AS offer_status
FROM applications a
JOIN offers f ON f.application_id = a.application_id
WHERE a.application_id = 1
 AND f.status = 'Accepted';
```

Query 119 - Placement Officer Withdraws Application on Behalf of Student

```
INSERT INTO application_status_history (application_id, old_status,
  new_status, changed_by, change_reason)
SELECT application_id, status, 'Withdrawn', 5, 'Withdrawn on written student
  request'
FROM applications
WHERE application_id = 4 AND status NOT IN ('Selected', 'Rejected',
  'Withdrawn');
UPDATE applications SET status = 'Withdrawn'
WHERE application_id = 4 AND status NOT IN ('Selected', 'Rejected',
  'Withdrawn');
```

Query 120 - View Withdrawn Applications

```
SELECT a.application_id, u.full_name AS student, o.title, c.company_name,
  a.applied_on, a.updated_at
```

```
FROM applications a
JOIN student_profiles s ON a.student_id = s.student_id
JOIN users u ON s.user_id = u.user_id
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
WHERE a.status = 'Withdrawn'
ORDER BY a.updated_at DESC;
```

Query 121 - Re-apply After Withdrawal Before Deadline

```
INSERT INTO application_status_history (application_id, old_status,
  new_status, changed_by, change_reason)
SELECT a.application_id, a.status, 'Applied', 2, 'Re-applied before deadline'
FROM applications a
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
WHERE a.application_id = 4 AND a.student_id = 1 AND a.status = 'Withdrawn'
 AND o.status = 'Open' AND o.application_deadline >= CURDATE();
UPDATE applications a
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
SET a.status = 'Applied'
WHERE a.application_id = 4 AND a.student_id = 1 AND a.status = 'Withdrawn'
 AND o.status = 'Open' AND o.application_deadline >= CURDATE();
```

Query 122 - View Withdrawal and Re-application History

```
SELECT history_id, application_id, old_status, new_status, changed_by,
  change_reason, changed_at
FROM application_status_history
WHERE application_id = 4
ORDER BY changed_at;
```

EP05 - INTERVIEW AND OFFER MANAGEMENT

TASK T17 - SCHEDULE INTERVIEW

Task ID	T17
User Story ID	EP05-US01
Task Title	Schedule Interview

DATABASE QUERIES

Query 123 - Create Interview Table

```
CREATE TABLE interviews (
 interview_id INT PRIMARY KEY AUTO_INCREMENT,
 application_id INT NOT NULL,
 round_number INT DEFAULT 1,
 interview_datetime DATETIME NOT NULL,
 mode VARCHAR(10) NOT NULL,
 venue_or_link VARCHAR(255),
 status VARCHAR(20) DEFAULT 'Scheduled',
 result VARCHAR(20) DEFAULT 'Pending',
 rescheduled_from DATETIME NULL,
 cancel_reason VARCHAR(255),
 scheduled_by INT NOT NULL,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
 FOREIGN KEY (application_id) REFERENCES applications(application_id),
 FOREIGN KEY (scheduled_by) REFERENCES users(user_id)
);
```

Query 124 - Schedule Interview for Shortlisted Applicant - TechNova

```
INSERT INTO interviews (application_id, round_number, interview_datetime,
  mode, venue_or_link, scheduled_by)
SELECT a.application_id, 1, '2026-10-28 10:00:00', 'Online',
  'https://meet.example.com/technova-r1', 4
FROM applications a
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
WHERE a.application_id = 1 AND c.user_id = 4
 AND a.status = 'Shortlisted'
 AND '2026-10-28 10:00:00' > CURRENT_TIMESTAMP;
INSERT INTO application_status_history (application_id, old_status,
  new_status, changed_by, change_reason)
SELECT application_id, status, 'Interview Scheduled', 4, 'Interview scheduled
  for round 1'
```

```
FROM applications WHERE application_id = 1 AND status = 'Shortlisted';
UPDATE applications SET status = 'Interview Scheduled'
WHERE application_id = 1 AND status = 'Shortlisted';
```

Query 125 - Schedule Interview for Shortlisted Applicant - BrightPath

```
INSERT INTO interviews (application_id, round_number, interview_datetime,
  mode, venue_or_link, scheduled_by)
SELECT a.application_id, 1, '2026-10-29 11:00:00', 'Offline', 'Seminar Hall,
  Main Campus', 7
FROM applications a
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
WHERE a.application_id = 3 AND c.user_id = 7
 AND a.status = 'Shortlisted'
 AND '2026-10-29 11:00:00' > CURRENT_TIMESTAMP;
INSERT INTO application_status_history (application_id, old_status,
  new_status, changed_by, change_reason)
SELECT application_id, status, 'Interview Scheduled', 7, 'Interview scheduled
  for round 1'
FROM applications WHERE application_id = 3 AND status = 'Shortlisted';
UPDATE applications SET status = 'Interview Scheduled'
WHERE application_id = 3 AND status = 'Shortlisted';
```

Query 126 - Check Interview Time Clash for a Student

```
SELECT i.interview_id, i.interview_datetime, i.venue_or_link
FROM interviews i
JOIN applications a ON i.application_id = a.application_id
WHERE a.student_id = 1
 AND i.interview_datetime = '2026-10-28 10:00:00'
 AND i.status = 'Scheduled';
```

Query 127 - Reschedule Interview

```
UPDATE interviews
SET rescheduled_from = interview_datetime,
 interview_datetime = '2026-10-30 11:00:00'
WHERE interview_id = 2
 AND status = 'Scheduled';
```

Query 128 - Record Interview Result

```
UPDATE interviews
SET result = 'Passed', status = 'Completed'
WHERE interview_id = 1
 AND status = 'Scheduled';
```

Query 129 - Cancel Interview with Reason (Administrator or Placement Officer)

```
UPDATE interviews
SET status = 'Cancelled', cancel_reason = 'Interview panel unavailable'
WHERE interview_id = 2
 AND status = 'Scheduled';
```

EP05 - INTERVIEW AND OFFER MANAGEMENT

TASK T18 - VIEW INTERVIEW SCHEDULE

Task ID	T18
User Story ID	EP05-US02
Task Title	View Interview Schedule

DATABASE QUERIES

Query 130 - Student Views Own Interviews

```
SELECT i.interview_id, o.title, c.company_name, i.round_number,
  i.interview_datetime, i.mode,
 i.venue_or_link, i.status, i.result,
 CASE WHEN i.interview_datetime >= CURRENT_TIMESTAMP THEN 'Upcoming' ELSE
   'Past' END AS timing
FROM interviews i
JOIN applications a ON i.application_id = a.application_id
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
WHERE a.student_id = 1
ORDER BY i.interview_datetime;
```

Query 131 - Company Recruiter Views Interviews of Own Opportunities

```
SELECT i.interview_id, u.full_name AS student, o.title, i.round_number,
  i.interview_datetime, i.mode,
 i.venue_or_link, i.status, i.result
FROM interviews i
JOIN applications a ON i.application_id = a.application_id
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
JOIN student_profiles s ON a.student_id = s.student_id
JOIN users u ON s.user_id = u.user_id
WHERE c.user_id = 4
ORDER BY i.interview_datetime;
```

Query 132 - Placement Officer Views Day-Wise Schedule with Venue Clashes

```
SELECT DATE(i.interview_datetime) AS interview_date, i.interview_datetime,
  i.venue_or_link,
 COUNT(*) AS interviews_at_same_time_and_venue
FROM interviews i
WHERE i.status = 'Scheduled'
GROUP BY DATE(i.interview_datetime), i.interview_datetime, i.venue_or_link
ORDER BY i.interview_datetime;
```

Query 133 - Faculty Coordinator Views Department Interviews (Read-Only)

```
SELECT u.full_name, o.title, c.company_name, i.interview_datetime, i.mode,
  i.status, i.result
FROM interviews i
JOIN applications a ON i.application_id = a.application_id
JOIN student_profiles s ON a.student_id = s.student_id
JOIN users u ON s.user_id = u.user_id
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
WHERE s.branch = 'CSE'
ORDER BY i.interview_datetime;
```

Query 134 - Administrator Searches Interviews by Date or Company

```
SELECT i.interview_id, u.full_name AS student, c.company_name, o.title,
  i.interview_datetime, i.status,
 i.result, i.created_at, i.updated_at
FROM interviews i
JOIN applications a ON i.application_id = a.application_id
JOIN student_profiles s ON a.student_id = s.student_id
JOIN users u ON s.user_id = u.user_id
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
WHERE DATE(i.interview_datetime) BETWEEN '2026-10-25' AND '2026-10-31'
 OR c.company_name LIKE '%TechNova%'
ORDER BY i.interview_datetime;
```

EP05 - INTERVIEW AND OFFER MANAGEMENT

TASK T19 - RELEASE OFFER

Task ID	T19
User Story ID	EP05-US03
Task Title	Release Offer

DATABASE QUERIES

Query 135 - Create Offer Table

```
CREATE TABLE offers (
 offer_id INT PRIMARY KEY AUTO_INCREMENT,
 application_id INT NOT NULL UNIQUE,
 job_title VARCHAR(150) NOT NULL,
 package_amount DECIMAL(10,2) NOT NULL,
 joining_date DATE,
 offer_validity_date DATE NOT NULL,
 status VARCHAR(20) DEFAULT 'Pending',
 remarks VARCHAR(255),
 released_by INT NOT NULL,
 responded_at TIMESTAMP NULL,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
 FOREIGN KEY (application_id) REFERENCES applications(application_id),
 FOREIGN KEY (released_by) REFERENCES users(user_id)
);
```

Query 136 - Release Offer to Selected Student - TechNova

```
INSERT INTO offers (application_id, job_title, package_amount, joining_date,
  offer_validity_date, released_by)
SELECT a.application_id, o.title, 18000, '2027-01-05', '2026-11-10', 4
FROM applications a
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
WHERE a.application_id = 1 AND c.user_id = 4
 AND a.status = 'Selected'
 AND NOT EXISTS (SELECT 1 FROM offers f WHERE f.application_id =
  a.application_id);
```

Query 137 - Release Offer to Selected Student - BrightPath

```
INSERT INTO offers (application_id, job_title, package_amount, joining_date,
  offer_validity_date, released_by)
SELECT a.application_id, o.title, 10000, '2027-01-10', '2026-11-12', 7
```

```
FROM applications a
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
WHERE a.application_id = 3 AND c.user_id = 7
 AND a.status = 'Selected'
 AND NOT EXISTS (SELECT 1 FROM offers f WHERE f.application_id =
   a.application_id);
```

Query 138 - List Selected Applications Without an Offer

```
SELECT a.application_id, u.full_name AS student, o.title, c.company_name
FROM applications a
JOIN student_profiles s ON a.student_id = s.student_id
JOIN users u ON s.user_id = u.user_id
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
LEFT JOIN offers f ON f.application_id = a.application_id
WHERE a.status = 'Selected'
 AND f.offer_id IS NULL;
```

Query 139 - Edit Offer Before Student Responds

```
UPDATE offers
SET package_amount = 20000, remarks = 'Stipend revised after management
  approval'
WHERE offer_id = 1
 AND status = 'Pending';
```

Query 140 - Revoke Offer with Reason (example for any pending offer)

```
UPDATE offers
SET status = 'Revoked', remarks = 'Offer withdrawn due to company hiring
  freeze'
WHERE offer_id = 3
 AND status = 'Pending';
```

Query 141 - View All Offers

```
SELECT f.offer_id, u.full_name AS student, c.company_name, f.job_title,
  f.package_amount, f.joining_date,
 f.offer_validity_date, f.status, f.created_at, f.updated_at
FROM offers f
JOIN applications a ON f.application_id = a.application_id
JOIN student_profiles s ON a.student_id = s.student_id
JOIN users u ON s.user_id = u.user_id
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
ORDER BY f.created_at DESC;
```

INDUSTRIAL ASSIGNMENT- 5

EP05 - INTERVIEW AND OFFER MANAGEMENT

TASK T20 - RESPOND TO OFFER

Task ID	T20
User Story ID	EP05-US04
Task Title	Respond to Offer

DATABASE QUERIES

Query 142 - Student Views Pending Offers

```
SELECT f.offer_id, c.company_name, f.job_title, f.package_amount,
  f.joining_date, f.offer_validity_date,
 f.status
FROM offers f
JOIN applications a ON f.application_id = a.application_id
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
WHERE a.student_id = 1
 AND f.status = 'Pending'
ORDER BY f.offer_validity_date;
```

Query 143 - Student Accepts Offer

```
UPDATE offers f
JOIN applications a ON f.application_id = a.application_id
SET f.status = 'Accepted', f.responded_at = CURRENT_TIMESTAMP
WHERE f.offer_id = 1
 AND a.student_id = 1
 AND f.status = 'Pending'
 AND f.offer_validity_date >= CURDATE();
```

Query 144 - Student Rejects Offer

```
UPDATE offers f
JOIN applications a ON f.application_id = a.application_id
SET f.status = 'Rejected', f.responded_at = CURRENT_TIMESTAMP
WHERE f.offer_id = 2
 AND a.student_id = 2
 AND f.status = 'Pending'
 AND f.offer_validity_date >= CURDATE();
```

Query 145 - Mark Pending Offers as Expired After Validity Date

```
UPDATE offers
SET status = 'Expired'
WHERE status = 'Pending'
```

```
 AND offer_validity_date < CURDATE();
```

Query 146 - Company Recruiter Views Responses to Own Offers

```
SELECT f.offer_id, u.full_name AS student, f.job_title, f.status,
  f.responded_at
FROM offers f
JOIN applications a ON f.application_id = a.application_id
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
JOIN student_profiles s ON a.student_id = s.student_id
JOIN users u ON s.user_id = u.user_id
WHERE c.user_id = 4
ORDER BY f.created_at DESC;
```

Query 147 - Placement Officer Views Offer Response Counts

```
SELECT c.company_name, f.status, COUNT(*) AS total_offers
FROM offers f
JOIN applications a ON f.application_id = a.application_id
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
GROUP BY c.company_name, f.status
ORDER BY c.company_name, f.status;
```

Query 148 - Administrator Changes Response in an Exceptional Case

```
UPDATE offers
SET status = 'Pending', responded_at = NULL,
 remarks = 'Response reopened by Administrator after written request'
WHERE offer_id = 2
 AND status = 'Rejected'
 AND offer_validity_date >= CURDATE();
```

EP06 - DASHBOARD AND REPORTS

TASK T21 - VIEW DASHBOARD

Task ID	T21
User Story ID	EP06-US01
Task Title	View Dashboard

DATABASE QUERIES

Query 149 - Create Dashboard Preferences Table

```
CREATE TABLE dashboard_preferences (
 preference_id INT PRIMARY KEY AUTO_INCREMENT,
 user_id INT NOT NULL,
 widget_name VARCHAR(100) NOT NULL,
 display_order INT DEFAULT 1,
 is_visible BOOLEAN DEFAULT TRUE,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
 UNIQUE (user_id, widget_name),
 FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

Query 150 - Insert Dashboard Widget Preferences

```
INSERT INTO dashboard_preferences (user_id, widget_name, display_order,
  is_visible)
VALUES (2, 'My Applications', 1, TRUE),
 (2, 'Upcoming Interviews', 2, TRUE),
 (2, 'Pending Offers', 3, TRUE),
 (2, 'New Opportunities', 4, TRUE),
 (4, 'Open Opportunities', 1, TRUE),
 (4, 'New Applicants', 2, TRUE);
```

Query 151 - Student Dashboard Summary

```
SELECT
 (SELECT COUNT(*) FROM applications WHERE student_id = 1) AS
   total_applications,
 (SELECT COUNT(*) FROM applications WHERE student_id = 1 AND status =
   'Shortlisted') AS shortlisted,
 (SELECT COUNT(*) FROM interviews i JOIN applications a ON i.application_id =
   a.application_id
 WHERE a.student_id = 1 AND i.interview_datetime >= CURRENT_TIMESTAMP) AS
   upcoming_interviews,
 (SELECT COUNT(*) FROM offers f JOIN applications a ON f.application_id =
```

```
  a.application_id
 WHERE a.student_id = 1 AND f.status = 'Pending') AS pending_offers,
 (SELECT COUNT(*) FROM opportunities WHERE status = 'Open') AS
  open_opportunities;
```

Query 152 - Company Recruiter Dashboard Summary

```
SELECT
 (SELECT COUNT(*) FROM opportunities o JOIN companies c ON o.company_id =
  c.company_id
 WHERE c.user_id = 4 AND o.status = 'Open') AS open_opportunities,
 (SELECT COUNT(*) FROM applications a JOIN opportunities o ON a.opportunity_id
  = o.opportunity_id
 JOIN companies c ON o.company_id = c.company_id
 WHERE c.user_id = 4) AS total_applicants,
 (SELECT COUNT(*) FROM applications a JOIN opportunities o ON a.opportunity_id
  = o.opportunity_id
 JOIN companies c ON o.company_id = c.company_id
 WHERE c.user_id = 4 AND a.status = 'Applied') AS new_applicants,
 (SELECT COUNT(*) FROM interviews i JOIN applications a ON i.application_id =
  a.application_id
 JOIN opportunities o ON a.opportunity_id = o.opportunity_id
 JOIN companies c ON o.company_id = c.company_id
 WHERE c.user_id = 4 AND i.status = 'Scheduled') AS interviews_due;
```

Query 153 - Placement Officer Dashboard Summary

```
SELECT
 (SELECT COUNT(*) FROM student_profiles) AS total_students,
 (SELECT COUNT(*) FROM companies) AS total_companies,
 (SELECT COUNT(*) FROM opportunities WHERE status = 'Open') AS
  open_opportunities,
 (SELECT COUNT(*) FROM applications) AS total_applications,
 (SELECT COUNT(*) FROM offers WHERE status = 'Accepted') AS accepted_offers,
 (SELECT COUNT(*) FROM users WHERE status = 'Pending') AS pending_approvals;
```

Query 154 - Faculty Coordinator Dashboard Summary (Department)

```
SELECT
 (SELECT COUNT(*) FROM student_profiles WHERE branch = 'CSE') AS
  department_students,
 (SELECT COUNT(*) FROM student_profiles WHERE branch = 'CSE'
 AND verification_status = 'Verified') AS verified_profiles,
 (SELECT COUNT(*) FROM applications a JOIN student_profiles s ON a.student_id
  = s.student_id
 WHERE s.branch = 'CSE') AS department_applications,
 (SELECT COUNT(DISTINCT a.student_id) FROM offers f JOIN applications a ON
  f.application_id = a.application_id
```

```
JOIN student_profiles s ON a.student_id = s.student_id
WHERE s.branch = 'CSE' AND f.status = 'Accepted') AS students_with_offer;
```

Query 155 - Administrator Dashboard Summary

```
SELECT
 (SELECT COUNT(*) FROM users) AS total_users,
 (SELECT COUNT(*) FROM users WHERE status = 'Pending') AS pending_accounts,
 (SELECT COUNT(*) FROM login_sessions WHERE is_active = TRUE) AS
   active_sessions,
 (SELECT COUNT(*) FROM opportunities) AS opportunity_records,
 (SELECT COUNT(*) FROM applications) AS application_records;
```

Query 156 - Update Widget Order and Visibility

```
UPDATE dashboard_preferences
SET display_order = 1, is_visible = TRUE
WHERE user_id = 2 AND widget_name = 'Upcoming Interviews';
UPDATE dashboard_preferences
SET display_order = 2
WHERE user_id = 2 AND widget_name = 'My Applications';
```

EP06 - DASHBOARD AND REPORTS

TASK T22 - VIEW OPPORTUNITY SUMMARY

Task ID	T22
User Story ID	EP06-US02
Task Title	View Opportunity Summary

DATABASE QUERIES

Query 157 - Recruiter Views Own Opportunity Summary

```
SELECT o.title, o.status, COUNT(a.application_id) AS total_applications
FROM opportunities o
JOIN companies c ON o.company_id = c.company_id
LEFT JOIN applications a ON a.opportunity_id = o.opportunity_id
WHERE c.user_id = 4
GROUP BY o.opportunity_id, o.title, o.status
ORDER BY o.title;
```

Query 158 - Opportunity Summary by Type and Status

```
SELECT type, status, COUNT(*) AS total_opportunities
FROM opportunities
GROUP BY type, status
ORDER BY type, status;
```

Query 159 - Company-Wise Opportunity Summary

```
SELECT c.company_name, COUNT(o.opportunity_id) AS total_opportunities,
 SUM(o.status = 'Open') AS open_opportunities, SUM(o.vacancies) AS
   total_vacancies
FROM companies c
LEFT JOIN opportunities o ON c.company_id = o.company_id
GROUP BY c.company_id, c.company_name
ORDER BY total_opportunities DESC;
```

Query 160 - Opportunity Summary for a Date Range

```
SELECT DATE(created_at) AS posted_on, COUNT(*) AS total_opportunities
FROM opportunities
WHERE created_at BETWEEN '2026-10-01 00:00:00' AND '2026-10-31 23:59:59'
GROUP BY DATE(created_at)
ORDER BY posted_on;
```

Query 161 - Student Views Count of Open Opportunities by Type

```
SELECT type, COUNT(*) AS open_opportunities
FROM opportunities
```

```
WHERE status = 'Open'
GROUP BY type;
```

Query 162 - Faculty Coordinator Views Opportunities Eligible for the Department Branch

```
SELECT o.opportunity_id, o.title, c.company_name, o.type, o.min_cgpa,
  o.application_deadline
FROM opportunities o
JOIN companies c ON o.company_id = c.company_id
WHERE o.status = 'Open'
 AND FIND_IN_SET('CSE', o.eligible_branches) > 0
ORDER BY o.application_deadline;
```

EP06 - DASHBOARD AND REPORTS

TASK T23 - VIEW PLACEMENT SUMMARY

Task ID	T23
User Story ID	EP06-US03
Task Title	View Placement Summary

DATABASE QUERIES

Query 163 - Student Views Own Placement Summary

```
SELECT
 (SELECT COUNT(*) FROM applications WHERE student_id = 1) AS applications,
 (SELECT COUNT(*) FROM applications WHERE student_id = 1 AND status =
   'Shortlisted') AS shortlisted,
 (SELECT COUNT(*) FROM interviews i JOIN applications a ON i.application_id =
   a.application_id
  WHERE a.student_id = 1) AS interviews,
 (SELECT COUNT(*) FROM offers f JOIN applications a ON f.application_id =
   a.application_id
  WHERE a.student_id = 1) AS offers,
 (SELECT COUNT(*) FROM offers f JOIN applications a ON f.application_id =
   a.application_id
  WHERE a.student_id = 1 AND f.status = 'Accepted') AS accepted_offers;
```

Query 164 - Branch-Wise Placement Summary

```
SELECT s.branch,
 COUNT(DISTINCT s.student_id) AS total_students,
 COUNT(DISTINCT CASE WHEN f.status = 'Accepted' THEN s.student_id END) AS
   students_with_offer,
 ROUND(100 * COUNT(DISTINCT CASE WHEN f.status = 'Accepted' THEN s.student_id
   END)
 / COUNT(DISTINCT s.student_id), 2) AS placement_percentage
FROM student_profiles s
LEFT JOIN applications a ON s.student_id = a.student_id
LEFT JOIN offers f ON a.application_id = f.application_id
GROUP BY s.branch;
```

Query 165 - Company-Wise Applications, Selections and Offers

```
SELECT c.company_name, COUNT(DISTINCT a.application_id) AS applications,
 SUM(a.status = 'Selected') AS selected, COUNT(DISTINCT f.offer_id) AS offers,
 SUM(f.status = 'Accepted') AS accepted_offers
FROM companies c
JOIN opportunities o ON c.company_id = o.company_id
LEFT JOIN applications a ON a.opportunity_id = o.opportunity_id
```

```
LEFT JOIN offers f ON f.application_id = a.application_id
GROUP BY c.company_id, c.company_name
ORDER BY c.company_name;
```

Query 166 - List of Placed Students (Accepted Offers)

```
SELECT u.full_name, s.enrollment_no, s.branch, c.company_name, o.type,
  f.job_title, f.package_amount
FROM offers f
JOIN applications a ON f.application_id = a.application_id
JOIN student_profiles s ON a.student_id = s.student_id
JOIN users u ON s.user_id = u.user_id
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
WHERE f.status = 'Accepted'
ORDER BY c.company_name, u.full_name;
```

Query 167 - Company Recruiter Views Own Company Placement Summary

```
SELECT a.status, COUNT(*) AS total_applications
FROM applications a
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
WHERE c.user_id = 4
GROUP BY a.status
ORDER BY a.status;
```

Query 168 - Faculty Coordinator Views Department Placement Summary for a Period

```
SELECT COUNT(DISTINCT a.student_id) AS students_applied,
 COUNT(DISTINCT CASE WHEN a.status = 'Selected' THEN a.student_id END) AS
   students_selected,
 COUNT(DISTINCT CASE WHEN f.status = 'Accepted' THEN a.student_id END) AS
   students_with_offer
FROM applications a
JOIN student_profiles s ON a.student_id = s.student_id
LEFT JOIN offers f ON f.application_id = a.application_id
WHERE s.branch = 'CSE'
 AND a.applied_on BETWEEN '2026-10-01' AND '2026-10-31';
```

EP06 - DASHBOARD AND REPORTS

TASK T24 - GENERATE REPORTS

Task ID	T24
User Story ID	EP06-US04
Task Title	Generate Reports

DATABASE QUERIES

Query 169 - Create Report History Table

```
CREATE TABLE report_history (
 report_id INT PRIMARY KEY AUTO_INCREMENT,
 report_type VARCHAR(100) NOT NULL,
 start_date DATE,
 end_date DATE,
 scope VARCHAR(100),
 export_format VARCHAR(20),
 generated_by INT NOT NULL,
 generated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 schedule_frequency VARCHAR(50),
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
 FOREIGN KEY (generated_by) REFERENCES users(user_id)
);
```

Query 170 - Insert Placement Report Request

```
INSERT INTO report_history (report_type, start_date, end_date, scope,
  export_format, generated_by)
VALUES ('Placement Summary Report', '2026-10-01', '2026-10-31', 'All
  Branches', 'PDF', 5);
```

Query 171 - Validate Report Date Range

```
SELECT report_id, report_type, start_date, end_date
FROM report_history
WHERE end_date < start_date;
```

Query 172 - Generate Placement Summary Report

```
SELECT s.branch, c.company_name, COUNT(DISTINCT a.application_id) AS
  applications,
 SUM(a.status = 'Selected') AS selected, SUM(f.status = 'Accepted') AS
   accepted_offers
FROM applications a
```

```
JOIN student_profiles s ON a.student_id = s.student_id
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
LEFT JOIN offers f ON f.application_id = a.application_id
WHERE a.applied_on BETWEEN '2026-10-01' AND '2026-10-31'
GROUP BY s.branch, c.company_name
ORDER BY s.branch, c.company_name;
```

Query 173 - Generate Opportunity Activity Report

```
SELECT o.type, o.status, COUNT(*) AS total_opportunities, SUM(o.vacancies) AS
  total_vacancies
FROM opportunities o
WHERE o.created_at BETWEEN '2026-10-01 00:00:00' AND '2026-10-31 23:59:59'
GROUP BY o.type, o.status
ORDER BY o.type, o.status;
```

Query 174 - Generate Application Status Report

```
SELECT a.status, COUNT(*) AS total_applications
FROM applications a
WHERE a.applied_on BETWEEN '2026-10-01' AND '2026-10-31'
GROUP BY a.status
ORDER BY total_applications DESC;
```

Query 175 - Generate Full System Activity Report

```
SELECT
 (SELECT COUNT(*) FROM users) AS total_users,
 (SELECT COUNT(*) FROM student_profiles) AS student_profiles,
 (SELECT COUNT(*) FROM companies) AS companies,
 (SELECT COUNT(*) FROM opportunities) AS opportunities,
 (SELECT COUNT(*) FROM applications) AS applications,
 (SELECT COUNT(*) FROM interviews) AS interviews,
 (SELECT COUNT(*) FROM offers) AS offers;
```

Query 176 - View Generated Report History

```
SELECT report_id, report_type, start_date, end_date, scope, export_format,
  generated_by, generated_at,
 schedule_frequency, created_at, updated_at
FROM report_history
ORDER BY generated_at DESC;
```

EP07 - SEARCH AND FILTER

TASK T25 - SEARCH OPPORTUNITIES

Task ID	T25
User Story ID	EP07-US01
Task Title	Search Opportunities

DATABASE QUERIES

Query 177 - Search Open Opportunities by Title

```
SELECT o.opportunity_id, o.title, c.company_name, o.type, o.location,
  o.min_cgpa, o.application_deadline, o.status
FROM opportunities o
JOIN companies c ON o.company_id = c.company_id
WHERE o.status = 'Open'
 AND o.title LIKE '%Developer%'
ORDER BY o.application_deadline;
```

Query 178 - Search Opportunities by Company Name

```
SELECT o.opportunity_id, o.title, c.company_name, o.type, o.location,
  o.min_cgpa, o.application_deadline, o.status
FROM opportunities o
JOIN companies c ON o.company_id = c.company_id
WHERE o.status = 'Open'
 AND c.company_name LIKE '%TechNova%'
ORDER BY o.application_deadline;
```

Query 179 - Search Opportunities by Keyword in Description or Location

```
SELECT o.opportunity_id, o.title, c.company_name, o.type, o.location,
  o.min_cgpa, o.application_deadline, o.status
FROM opportunities o
JOIN companies c ON o.company_id = c.company_id
WHERE o.status = 'Open'
 AND (o.description LIKE '%React%' OR o.location LIKE '%Bengaluru%')
ORDER BY o.application_deadline;
```

Query 180 - Recruiter Searches Own Opportunities

```
SELECT o.opportunity_id, o.title, c.company_name, o.type, o.location,
  o.min_cgpa, o.application_deadline, o.status
FROM opportunities o
JOIN companies c ON o.company_id = c.company_id
WHERE c.user_id = 4
 AND o.title LIKE '%Intern%'
```

```
ORDER BY o.created_at DESC;
```

Query 181 - Placement Officer or Administrator Searches All Opportunities Including Closed

```
SELECT o.opportunity_id, o.title, c.company_name, o.type, o.location,
  o.min_cgpa, o.application_deadline, o.status
FROM opportunities o
JOIN companies c ON o.company_id = c.company_id
WHERE o.title LIKE '%Developer%'
 OR c.company_name LIKE '%Bright%'
ORDER BY o.status, o.application_deadline;
```

EP07 - SEARCH AND FILTER

TASK T26 - FILTER OPPORTUNITIES

Task ID	T26
User Story ID	EP07-US02
Task Title	Filter Opportunities

DATABASE QUERIES

Query 182 - Filter Opportunities by Type

```
SELECT o.opportunity_id, o.title, c.company_name, o.type, o.location,
  o.min_cgpa, o.application_deadline, o.status
FROM opportunities o
JOIN companies c ON o.company_id = c.company_id
WHERE o.status = 'Open'
 AND o.type = 'Internship'
ORDER BY o.application_deadline;
```

Query 183 - Filter Opportunities the Student Is Eligible For

```
SELECT o.opportunity_id, o.title, c.company_name, o.type, o.location,
  o.min_cgpa, o.application_deadline, o.status
FROM opportunities o
JOIN companies c ON o.company_id = c.company_id
JOIN student_profiles s ON s.student_id = 2
WHERE o.status = 'Open'
 AND o.application_deadline >= CURDATE()
 AND s.cgpa >= o.min_cgpa
 AND s.backlogs <= o.max_backlogs
 AND s.semester >= o.min_semester
 AND FIND_IN_SET(s.branch, o.eligible_branches) > 0
ORDER BY o.application_deadline;
```

Query 184 - Filter Opportunities by Deadline Range

```
SELECT o.opportunity_id, o.title, c.company_name, o.type, o.location,
  o.min_cgpa, o.application_deadline, o.status
FROM opportunities o
JOIN companies c ON o.company_id = c.company_id
WHERE o.application_deadline BETWEEN '2026-11-01' AND '2026-12-31'
ORDER BY o.application_deadline;
```

Query 185 - Filter Opportunities by Status and Company

```
SELECT o.opportunity_id, o.title, c.company_name, o.type, o.location,
  o.min_cgpa, o.application_deadline, o.status
```

```
FROM opportunities o
JOIN companies c ON o.company_id = c.company_id
WHERE o.status IN ('Open', 'Closed')
 AND c.company_id = 1
ORDER BY o.status, o.application_deadline;
```

Query 186 - Combine Opportunity Filters Using AND Logic

```
SELECT o.opportunity_id, o.title, c.company_name, o.type, o.location,
  o.min_cgpa, o.application_deadline, o.status
FROM opportunities o
JOIN companies c ON o.company_id = c.company_id
WHERE o.status = 'Open'
 AND o.type = 'Placement'
 AND o.min_cgpa <= 8.00
 AND o.location LIKE '%Hyderabad%'
 AND o.application_deadline BETWEEN '2026-11-01' AND '2026-12-31'
ORDER BY o.application_deadline;
```

EP07 - SEARCH AND FILTER

TASK T27 - SEARCH STUDENT RECORDS

Task ID	T27
User Story ID	EP07-US03
Task Title	Search Student Records

DATABASE QUERIES

Query 187 - Search Students by Name

```
SELECT s.student_id, u.full_name, s.enrollment_no, s.branch, s.semester,
  s.cgpa, s.verification_status
FROM student_profiles s
JOIN users u ON s.user_id = u.user_id
WHERE u.full_name LIKE '%Priya%'
ORDER BY u.full_name;
```

Query 188 - Search Students by Enrollment Number

```
SELECT s.student_id, u.full_name, s.enrollment_no, s.branch, s.semester,
  s.cgpa, s.verification_status
FROM student_profiles s
JOIN users u ON s.user_id = u.user_id
WHERE s.enrollment_no = 'DCSE2023002';
```

Query 189 - Search Students by Branch and Semester

```
SELECT s.student_id, u.full_name, s.enrollment_no, s.branch, s.semester,
  s.cgpa, s.verification_status
FROM student_profiles s
JOIN users u ON s.user_id = u.user_id
WHERE s.branch = 'CSE'
 AND s.semester = 5
ORDER BY s.enrollment_no;
```

Query 190 - Company Recruiter Searches Among Own Applicants Only

```
SELECT DISTINCT s.student_id, u.full_name, s.branch, s.semester, s.cgpa
FROM applications a
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
JOIN student_profiles s ON a.student_id = s.student_id
JOIN users u ON s.user_id = u.user_id
WHERE c.user_id = 4
 AND u.full_name LIKE '%Shah%'
ORDER BY u.full_name;
```

Query 191 - Faculty Coordinator Searches Department Students with Masked Contact

```
SELECT s.student_id, u.full_name, s.enrollment_no, s.semester, s.cgpa,
 CONCAT('******', RIGHT(u.phone, 4)) AS masked_phone
FROM student_profiles s
JOIN users u ON s.user_id = u.user_id
WHERE s.branch = 'CSE'
 AND (u.full_name LIKE '%Rohan%' OR s.enrollment_no LIKE 'DCSE2023%')
ORDER BY s.enrollment_no;
```

EP07 - SEARCH AND FILTER

TASK T28 - FILTER APPLICATIONS

Task ID	T28
User Story ID	EP07-US04
Task Title	Filter Applications

DATABASE QUERIES

Query 192 - Filter Applications by Status

```
SELECT a.application_id, u.full_name AS student, s.branch, s.cgpa,
  c.company_name, o.title, a.applied_on, a.status
FROM applications a
JOIN student_profiles s ON a.student_id = s.student_id
JOIN users u ON s.user_id = u.user_id
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
WHERE a.status = 'Shortlisted'
ORDER BY a.applied_on DESC;
```

Query 193 - Filter Applications by Company or Opportunity

```
SELECT a.application_id, u.full_name AS student, s.branch, s.cgpa,
  c.company_name, o.title, a.applied_on, a.status
FROM applications a
JOIN student_profiles s ON a.student_id = s.student_id
JOIN users u ON s.user_id = u.user_id
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
WHERE c.company_name = 'TechNova Solutions'
 OR o.opportunity_id = 3
ORDER BY a.applied_on DESC;
```

Query 194 - Filter Applications by Branch and CGPA

```
SELECT a.application_id, u.full_name AS student, s.branch, s.cgpa,
  c.company_name, o.title, a.applied_on, a.status
FROM applications a
JOIN student_profiles s ON a.student_id = s.student_id
JOIN users u ON s.user_id = u.user_id
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
WHERE s.branch = 'CSE'
 AND s.cgpa >= 7.50
ORDER BY s.cgpa DESC;
```

Query 195 - Filter Applications by Date Range

```
SELECT a.application_id, u.full_name AS student, s.branch, s.cgpa,
  c.company_name, o.title, a.applied_on, a.status
FROM applications a
JOIN student_profiles s ON a.student_id = s.student_id
JOIN users u ON s.user_id = u.user_id
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
WHERE a.applied_on BETWEEN '2026-10-01' AND '2026-10-31'
ORDER BY a.applied_on;
```

Query 196 - Combine Application Filters Using AND Logic

```
SELECT a.application_id, u.full_name AS student, s.branch, s.cgpa,
  c.company_name, o.title, a.applied_on, a.status
FROM applications a
JOIN student_profiles s ON a.student_id = s.student_id
JOIN users u ON s.user_id = u.user_id
JOIN opportunities o ON a.opportunity_id = o.opportunity_id
JOIN companies c ON o.company_id = c.company_id
WHERE a.status IN ('Applied', 'Shortlisted', 'Interview Scheduled')
 AND c.company_name = 'TechNova Solutions'
 AND s.branch = 'CSE'
 AND s.cgpa >= 7.00
 AND a.applied_on BETWEEN '2026-10-01' AND '2026-10-31'
ORDER BY a.applied_on DESC;
```

END OF DATABASE QUERIES

All table definitions in this document include created_at and updated_at as TIMESTAMP attributes.

ENTITY & ATTRIBUTE DICTIONARY

STUDENT INTERNSHIP AND PLACEMENT MANAGEMENT PORTAL

Document Scope

This section identifies every entity created by a CREATE TABLE statement in the Database Queries above and lists its attributes, MySQL data types, declared constraints, and explicitly declared foreign-key references. Every table includes created_at and updated_at timestamp attributes.

ENTITY LIST

S.No.	Entity / Table	Episode
1	users	EP01 - USER MANAGEMENT
2	login_sessions	EP01 - USER MANAGEMENT
3	user_access_history	EP01 - USER MANAGEMENT
4	student_profiles	EP02 - STUDENT PROFILE MANAGEMENT
5	student_resumes	EP02 - STUDENT PROFILE MANAGEMENT
6	student_edit_history	EP02 - STUDENT PROFILE MANAGEMENT
7	companies	EP03 - COMPANY AND OPPORTUNITY MANAGEMENT
8	opportunities	EP03 - COMPANY AND OPPORTUNITY MANAGEMENT
9	opportunity_edit_history	EP03 - COMPANY AND OPPORTUNITY MANAGEMENT
10	opportunity_deletion_history	EP03 - COMPANY AND OPPORTUNITY MANAGEMENT
11	applications	EP04 - APPLICATION MANAGEMENT
12	application_status_history	EP04 - APPLICATION MANAGEMENT
13	interviews	EP05 - INTERVIEW AND OFFER MANAGEMENT
14	offers	EP05 - INTERVIEW AND OFFER MANAGEMENT
15	dashboard_preferences	EP06 - DASHBOARD AND REPORTS
16	report_history	EP06 - DASHBOARD AND REPORTS

INDUSTRIAL ASSIGNMENT- 5

Constraint notation: PK = Primary Key. FK = Foreign Key. The Foreign Key Reference column names the referenced table primary key only where the CREATE TABLE statement explicitly declares a FOREIGN KEY constraint.

EP01 - USER MANAGEMENT

ENTITY 01 - users

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
1	user_id	INT	PRIMARY KEY; AUTO_INCREMENT	-
2	full_name	VARCHAR(100)	NOT NULL	-
3	email	VARCHAR(100)	NOT NULL; UNIQUE	-
4	phone	VARCHAR(15)	NOT NULL; UNIQUE	-
5	password_hash	VARCHAR(255)	NOT NULL	-
6	role	VARCHAR(30)	NOT NULL	-
7	status	VARCHAR(20)	DEFAULT 'Pending'	-
8	failed_attempts	INT	DEFAULT 0	-
9	locked_until	TIMESTAMP	NULL allowed	-
10	address	VARCHAR(255)	NULL allowed	-
11	employee_id	VARCHAR(50)	NULL allowed	-
12	designation	VARCHAR(100)	NULL allowed	-
13	department	VARCHAR(100)	NULL allowed	-
14	joining_date	DATE	NULL allowed	-
15	enrollment_no	VARCHAR(30)	UNIQUE	-
16	branch	VARCHAR(50)	NULL allowed	-
17	semester	INT	NULL allowed	-
18	company_name	VARCHAR(150)	NULL allowed	-
19	company_website	VARCHAR(150)	NULL allowed	-
20	appointment_date	DATE	NULL allowed	-
21	scope_of_access	VARCHAR(255)	NULL allowed	-
22	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
23	updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP; ON UPDATE CURRENT_TIMESTAMP	-

EP01 - USER MANAGEMENT

ENTITY 02 - login_sessions

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
1	session_id	INT	PRIMARY KEY; AUTO_INCREMENT	-
2	user_id	INT	NOT NULL; FOREIGN KEY	users.user_id (PK)
3	session_token	VARCHAR(255)	NOT NULL; UNIQUE	-
4	login_time	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-
5	last_activity	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-
6	logout_time	TIMESTAMP	NULL allowed	-
7	is_active	BOOLEAN	DEFAULT TRUE	-
8	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-
9	updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP; ON UPDATE CURRENT_TIMESTAMP	-

EP01 - USER MANAGEMENT

ENTITY 03 - user_access_history

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
1	history_id	INT	PRIMARY KEY; AUTO_INCREMENT	-

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
2	user_id	INT	NOT NULL; FOREIGN KEY	users.user_id (PK)
3	action_type	VARCHAR(30)	NOT NULL	-
4	old_role	VARCHAR(30)	NULL allowed	-
5	new_role	VARCHAR(30)	NULL allowed	-
6	old_status	VARCHAR(20)	NULL allowed	-
7	new_status	VARCHAR(20)	NULL allowed	-
8	reason	VARCHAR(255)	NULL allowed	-
9	action_by	INT	FOREIGN KEY	users.user_id (PK)
10	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-
11	updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP; ON UPDATE CURRENT_TIMESTAMP	-

EP02 - STUDENT PROFILE MANAGEMENT

ENTITY 04 - student_profiles

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
1	student_id	INT	PRIMARY KEY; AUTO_INCREMENT	-
2	user_id	INT	NOT NULL; UNIQUE; FOREIGN KEY	users.user_id (PK)
3	enrollment_no	VARCHAR(30)	NOT NULL; UNIQUE	-
4	branch	VARCHAR(50)	NOT NULL	-
5	semester	INT	NOT NULL	-
6	cgpa	DECIMAL(4,2)	NOT NULL	-
7	backlogs	INT	DEFAULT 0	-
8	skills	VARCHAR(255)	NULL allowed	-
9	address	VARCHAR(255)	NULL allowed	-

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
10	date_of_birth	DATE	NULL allowed	-
11	linkedin_url	VARCHAR(150)	NULL allowed	-
12	github_url	VARCHAR(150)	NULL allowed	-
13	verification_status	VARCHAR(30)	DEFAULT 'Pending'	-
14	verification_remarks	VARCHAR(255)	NULL allowed	-
15	verified_by	INT	FOREIGN KEY	users.user_id (PK)
16	verified_at	TIMESTAMP	NULL allowed	-
17	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-
18	updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP; ON UPDATE CURRENT_TIMESTAMP	-

EP02 - STUDENT PROFILE MANAGEMENT

ENTITY 05 - student_resumes

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
1	resume_id	INT	PRIMARY KEY; AUTO_INCREMENT	-
2	student_id	INT	NOT NULL; FOREIGN KEY	student_profiles.student_id (PK)
3	file_name	VARCHAR(150)	NOT NULL	-
4	file_path	VARCHAR(255)	NOT NULL	-
5	file_type	VARCHAR(10)	NOT NULL	-
6	file_size_kb	INT	NOT NULL	-
7	is_current	BOOLEAN	DEFAULT TRUE	-
8	removal_reason	VARCHAR(255)	NULL allowed	-
9	removed_by	INT	FOREIGN KEY	users.user_id (PK)
10	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
11	updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP; ON UPDATE CURRENT_TIMESTAMP	-

EP02 - STUDENT PROFILE MANAGEMENT

ENTITY 06 - student_edit_history

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
1	history_id	INT	PRIMARY KEY; AUTO_INCREMENT	-
2	student_id	INT	NOT NULL; FOREIGN KEY	student_profiles.student_id (PK)
3	field_name	VARCHAR(100)	NOT NULL	-
4	old_value	VARCHAR(255)	NULL allowed	-
5	new_value	VARCHAR(255)	NULL allowed	-
6	reason	VARCHAR(255)	NULL allowed	-
7	edited_by	INT	NOT NULL; FOREIGN KEY	users.user_id (PK)
8	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-
9	updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP; ON UPDATE CURRENT_TIMESTAMP	-

EP03 - COMPANY AND OPPORTUNITY MANAGEMENT

ENTITY 07 - companies

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
1	company_id	INT	PRIMARY KEY; AUTO_INCREMENT	-
2	user_id	INT	NOT NULL; UNIQUE; FOREIGN KEY	users.user_id (PK)
3	company_name	VARCHAR(150)	NOT NULL	-

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
4	industry	VARCHAR(100)	NULL allowed	-
5	website	VARCHAR(150)	NULL allowed	-
6	location	VARCHAR(100)	NULL allowed	-
7	hr_name	VARCHAR(100)	NULL allowed	-
8	hr_email	VARCHAR(100)	NULL allowed	-
9	hr_phone	VARCHAR(15)	NULL allowed	-
10	description	TEXT	NULL allowed	-
11	company_size	VARCHAR(30)	NULL allowed	-
12	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-
13	updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP; ON UPDATE CURRENT_TIMESTAMP	-

EP03 - COMPANY AND OPPORTUNITY MANAGEMENT

ENTITY 08 - opportunities

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
1	opportunity_id	INT	PRIMARY KEY; AUTO_INCREMENT	-
2	company_id	INT	NOT NULL; FOREIGN KEY	companies.company_id (PK)
3	title	VARCHAR(150)	NOT NULL	-
4	type	VARCHAR(20)	NOT NULL	-
5	description	TEXT	NULL allowed	-
6	location	VARCHAR(100)	NULL allowed	-
7	min_cgpa	DECIMAL(4,2)	DEFAULT 0	-
8	eligible_branches	VARCHAR(100)	NULL allowed	-
9	min_semester	INT	DEFAULT 1	-
10	max_backlogs	INT	DEFAULT 0	-

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
11	compensation	DECIMAL(10,2)	NULL allowed	-
12	vacancies	INT	DEFAULT 1	-
13	application_deadline	DATE	NOT NULL	-
14	status	VARCHAR(20)	DEFAULT 'Open'	-
15	posted_by	INT	NOT NULL; FOREIGN KEY	users.user_id (PK)
16	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-
17	updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP; ON UPDATE CURRENT_TIMESTAMP	-

EP03 - COMPANY AND OPPORTUNITY MANAGEMENT

ENTITY 09 - opportunity_edit_history

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
1	history_id	INT	PRIMARY KEY; AUTO_INCREMENT	-
2	opportunity_id	INT	NOT NULL; FOREIGN KEY	opportunities.opportunity_id (PK)
3	field_name	VARCHAR(100)	NOT NULL	-
4	old_value	VARCHAR(255)	NULL allowed	-
5	new_value	VARCHAR(255)	NULL allowed	-
6	reason	VARCHAR(255)	NULL allowed	-
7	edited_by	INT	NOT NULL; FOREIGN KEY	users.user_id (PK)
8	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-
9	updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP; ON UPDATE CURRENT_TIMESTAMP	-

EP03 - COMPANY AND OPPORTUNITY MANAGEMENT

ENTITY 10 - opportunity_deletion_history

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
1	deletion_id	INT	PRIMARY KEY; AUTO_INCREMENT	-
2	opportunity_id	INT	NOT NULL; FOREIGN KEY	opportunities.opportunity_id (PK)
3	title	VARCHAR(150)	NOT NULL	-
4	previous_status	VARCHAR(20)	NULL allowed	-
5	reason	VARCHAR(255)	NOT NULL	-
6	deleted_by	INT	NOT NULL; FOREIGN KEY	users.user_id (PK)
7	deleted_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-
8	restored_at	TIMESTAMP	NULL allowed	-
9	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-
10	updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP; ON UPDATE CURRENT_TIMESTAMP	-

EP04 - APPLICATION MANAGEMENT

ENTITY 11 - applications

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
1	application_id	INT	PRIMARY KEY; AUTO_INCREMENT	-
2	student_id	INT	NOT NULL; FOREIGN KEY	student_profiles.student_id (PK)
3	opportunity_id	INT	NOT NULL; FOREIGN KEY	opportunities.opportunity_id (PK)
4	resume_id	INT	FOREIGN KEY	student_resumes.resume_id (PK)
5	applied_on	DATE	NOT NULL	-
6	status	VARCHAR(30)	DEFAULT 'Applied'	-

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
7	applied_by	INT	FOREIGN KEY	users.user_id (PK)
8	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-
9	updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP; ON UPDATE CURRENT_TIMESTAMP	-

Table-level constraint: UNIQUE (student_id, opportunity_id).

EP04 - APPLICATION MANAGEMENT

ENTITY 12 - application_status_history

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
1	history_id	INT	PRIMARY KEY; AUTO_INCREMENT	-
2	application_id	INT	NOT NULL; FOREIGN KEY	applications.application_id (PK)
3	old_status	VARCHAR(30)	NULL allowed	-
4	new_status	VARCHAR(30)	NOT NULL	-
5	changed_by	INT	NOT NULL; FOREIGN KEY	users.user_id (PK)
6	change_reason	VARCHAR(255)	NULL allowed	-
7	changed_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-
8	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-
9	updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP; ON UPDATE CURRENT_TIMESTAMP	-

EP05 - INTERVIEW AND OFFER MANAGEMENT

ENTITY 13 - interviews

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
1	interview_id	INT	PRIMARY KEY; AUTO_INCREMENT	-
2	application_id	INT	NOT NULL; FOREIGN KEY	applications.application_id (PK)
3	round_number	INT	DEFAULT 1	-
4	interview_datetime	DATETIME	NOT NULL	-
5	mode	VARCHAR(10)	NOT NULL	-
6	venue_or_link	VARCHAR(255)	NULL allowed	-
7	status	VARCHAR(20)	DEFAULT 'Scheduled'	-
8	result	VARCHAR(20)	DEFAULT 'Pending'	-
9	rescheduled_from	DATETIME	NULL allowed	-
10	cancel_reason	VARCHAR(255)	NULL allowed	-
11	scheduled_by	INT	NOT NULL; FOREIGN KEY	users.user_id (PK)
12	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-
13	updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP; ON UPDATE CURRENT_TIMESTAMP	-

EP05 - INTERVIEW AND OFFER MANAGEMENT

ENTITY 14 - offers

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
1	offer_id	INT	PRIMARY KEY; AUTO_INCREMENT	-
2	application_id	INT	NOT NULL; UNIQUE; FOREIGN KEY	applications.application_id (PK)
3	job_title	VARCHAR(150)	NOT NULL	-
4	package_amount	DECIMAL(10,2)	NOT NULL	-
5	joining_date	DATE	NULL allowed	-

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
6	offer_validity_date	DATE	NOT NULL	-
7	status	VARCHAR(20)	DEFAULT 'Pending'	-
8	remarks	VARCHAR(255)	NULL allowed	-
9	released_by	INT	NOT NULL; FOREIGN KEY	users.user_id (PK)
10	responded_at	TIMESTAMP	NULL allowed	-
11	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-
12	updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP; ON UPDATE CURRENT_TIMESTAMP	-

EP06 - DASHBOARD AND REPORTS

ENTITY 15 - dashboard_preferences

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
1	preference_id	INT	PRIMARY KEY; AUTO_INCREMENT	-
2	user_id	INT	NOT NULL; FOREIGN KEY	users.user_id (PK)
3	widget_name	VARCHAR(100)	NOT NULL	-
4	display_order	INT	DEFAULT 1	-
5	is_visible	BOOLEAN	DEFAULT TRUE	-
6	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-
7	updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP; ON UPDATE CURRENT_TIMESTAMP	-

Table-level constraint: UNIQUE (user_id, widget_name).

EP06 - DASHBOARD AND REPORTS

ENTITY 16 - report_history

S.No.	Attribute	Data Type	Constraints	Foreign Key Reference
1	report_id	INT	PRIMARY KEY; AUTO_INCREMENT	-
2	report_type	VARCHAR(100)	NOT NULL	-
3	start_date	DATE	NULL allowed	-
4	end_date	DATE	NULL allowed	-
5	scope	VARCHAR(100)	NULL allowed	-
6	export_format	VARCHAR(20)	NULL allowed	-
7	generated_by	INT	NOT NULL; FOREIGN KEY	users.user_id (PK)
8	generated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-
9	schedule_frequency	VARCHAR(50)	NULL allowed	-
10	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-
11	updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP; ON UPDATE CURRENT_TIMESTAMP	-